
训练计算最优的大语言模型

Jordan Hoffmann★、Sebastian Borgeaud★、Arthur Mensch★、Elena Buchatskaya、Trevor Cai、Eliza Rutherford、Diego de Las Casas、Lisa Anne Hendricks、Johannes Welbl、Aidan Clark、Tom Hennigan、Eric Noland、Katie Millican、George van den Driessche、Bogdan Damoc、Aurelia Guy、Simon Osindero、Karen Simonyan、Erich Elsen、Jack W. Rae、Oriol Vinyals 和 Laurent Sifre★

★ 同等贡献

通信作者：{jordanhoffmann|sborgeaud|amensch|sifre}@deepmind.com

© 2023 DeepMind。

保留所有权利。

arXiv:2203.15556v1 [cs.CL] 29 Mar 2022

摘要

我们研究了在给定计算预算下训练 Transformer 语言模型时的最优模型规模和 token 数量。我们发现，目前的大语言模型训练显著不足，这是近来在保持训练数据量不变的同时专注于扩大语言模型规模所造成的结果。通过用 50 亿到 5000 亿 token 训练 400 多个参数量从 7000 万到超过 160 亿的语言模型，我们发现，对于计算最优训练，模型规模与训练 token 数量应当等比例缩放：模型规模每翻一番，训练 token 数量也应翻一番。我们通过训练一个预测为计算最优的模型 Chinchilla 来检验这一假设；它采用与 Gopher 相同的计算预算，但有 70B 参数，并使用 4× 更多的数据。在广泛的下游评估任务上，Chinchilla 全面且显著地优于 Gopher (280B)、GPT-3 (175B)、Jurassic-1 (178B) 和 Megatron-Turing NLG (530B)。这也意味着，Chinchilla 用于 fine-tuning 和 inference 的计算量显著更少，从而极大地方便了下游使用。尤其值得一提的是，Chinchilla 在 MMLU 基准上达到当前最佳的 67.5% 平均准确率，比 Gopher 提高了 7% 以上。

目录

1. 引言	1
2. 相关工作	2
3. 估计参数量 / 训练 token 数量的最优分配	3
3.1. 方法 1: 固定模型规模, 改变训练 token 数量	3
3.2. 方法 2: 等 FLOP 剖面	4
3.3. 方法 3: 拟合参数化损失函数	5
3.4. 最优模型缩放	6
4. Chinchilla	7
4.1. 模型与训练细节	7
4.2. 结果	7
4.2.1. 语言建模	8
4.2.2. MMLU	8
4.2.3. 阅读理解	9
4.2.4. BIG-bench	9
4.2.5. 常识	10
4.2.6. 闭卷问答	10
4.2.7. 性别偏见与毒性	11
5. 讨论与结论	12
6. 致谢	12
参考文献	12
附录	15
A. 训练数据集	15
B. 最优余弦周期长度	15
C. 不同数据集上的缩放结果一致性	16
D. 缩放分析的细节	17
D.1. 方法 1: 固定模型规模并改变训练序列数	17
D.2. 方法 3: 对损失进行参数化拟合	17
D.3. 三种方法的预测计算最优前沿	18
D.4. 与 Kaplan et al. (2020) 的小规模比较	19
E. FLOP-损失前沿的曲率	20
F. FLOPs 计算	21
G. Chinchilla 与 Gopher 之间的其他差异	22
H. 结果	23
H.1. The Pile	23
H.2. MMLU	23
H.3. Winogender 设置	24
H.4. BIG-bench	24
I. 模型卡	25
J. 已训练模型列表	26

1. 引言

近来，一系列大语言模型（LLM）相继问世（Brown et al., 2020；Lieber et al., 2021；Rae et al., 2021；Smith et al., 2022；Thoppilan et al., 2022），其中最大的稠密语言模型目前已拥有超过 5000 亿个参数。这些大型 autoregressive Transformer（Vaswani et al., 2017）采用 zero-shot、few-shot 和 fine-tuning 等多种评估协议，在许多任务上展现出了令人瞩目的性能。

训练大语言模型的计算与能源成本十分可观（Rae et al., 2021；Thoppilan et al., 2022），并且会随模型规模增大而上升。在实践中，所分配的训练计算预算通常可以提前确定：有多少加速器可用，以及我们希望使用它们多长时间。由于这些大型模型通常只具备训练一次的可行性，因此，准确估计给定计算预算下的最佳模型超参数至关重要（Tay et al., 2021）。

Kaplan et al.（2020）表明，autoregressive 语言模型（LM）的参数数量与其性能之间存在幂律关系。因此，该领域一直在训练越来越大的模型，并期待性能得到提升。Kaplan et al.（2020）的一个重要结论是，为达到计算最优，不应把大型模型训练到其可能达到的最低损失。虽然我们得出了相同结论，但我们的估计表明，大型模型应使用远多于这些作者所建议的训练 token 来训练。具体而言，在计算预算增加 $10\times$ 时，他们建议模型规模增加 $5.5\times$ ，而训练 token 数量仅增加 $1.8\times$ 。与此不同，我们发现模型规模与训练 token 数量应按相同比例缩放。

沿用 Kaplan et al.（2020）和 GPT-3（Brown et al., 2020）的训练设置，近来训练的许多大型模型都使用了约 3000 亿 token（表 1），这与增加计算量时主要扩大模型规模的做法一致。

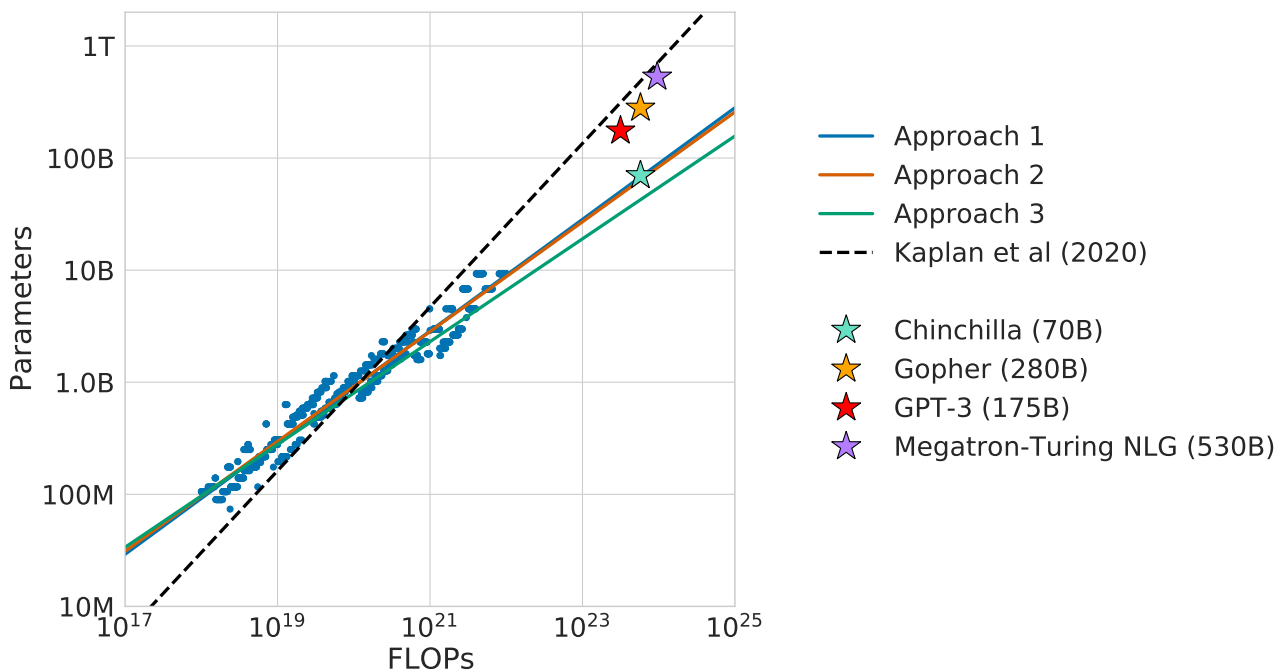


图 1 | 叠加的预测。我们叠加展示了三种不同方法的预测，以及 Kaplan et al. (2020) 的推算结果。我们发现，三种方法都预测当前的大型模型应当小得多，因此其训练时间应当比目前做法长得多。在图 A3 中，我们针对固定 FLOP 预算，将预测的最优 token 数量相对于最优参数数量作图。**Chinchilla 的表现优于 Gopher 和其他大型模型**（见第 4.2 节）。

模型	规模（参数数量）	训练 token
LaMDA (Thoppilan et al., 2022)	1370 亿	1680 亿
GPT-3 (Brown et al., 2020)	1750 亿	3000 亿
Jurassic (Lieber et al., 2021)	1780 亿	3000 亿
Gopher (Rae et al., 2021)	2800 亿	3000 亿
MT-NLG 530B (Smith et al., 2022)	5300 亿	2700 亿
Chinchilla	700 亿	1.4 万亿

表 1 | 当前的 LLM。我们列出了当前规模最大的五个稠密 Transformer 模型、它们的规模以及训练 token 数量。除 LaMDA (Thoppilan et al., 2022) 外，大多数模型都使用约 3000 亿 token 进行训练。我们提出 Chinchilla——一个小得多、但训练所用 token 远多于 300B 的模型。

在本研究中，我们重新审视这样一个问题：给定固定的 FLOPs 预算，¹ 应如何在模型规模与训练 token 数量之间进行权衡？为回答这一问题，我们将最终 pre-training 损失² $L(N, D)$ 建模为模型参数数量 N 和训练 token 数量 D 的函数。由于计算预算 C 是已见训练 token 数量和模型参数数量的确定性函数 $FLOPs(N, D)$ ，我们希望在约束 $FLOPs(N, D) = C$ 下最小化 L ：

$$N_{\text{opt}}(C), D_{\text{opt}}(C) = \arg \min_{N, D \text{ s.t. } FLOPs(N, D) = C} L(N, D). \quad (1)$$

函数 $N_{\text{opt}}(C)$ 和 $D_{\text{opt}}(C)$ 描述了计算预算 C 的最优分配。我们根据 400 多个模型的损失对这些函数进行了经验估计；这些模型的参数量从不足 70M 到超过 16B，并使用 5B 到超过 400B 的 token 进行训练，而且每种模型配置都采用了数种不同的训练时长。我们的方法所得结果与 Kaplan et al. (2020) 显著不同。图 1 突出展示了我们的结果，第 2 节则说明了我们的方法有何不同。

根据估计得到的计算最优前沿，我们预测：对于训练 Gopher 所用的计算预算，最优模型应当小 4 倍，同时用多 4 倍的 token 进行训练。我们通过在 1.4 万亿 token 上训练一个更接近计算最优的 70B 模型来验证这一点，并将该模型称为 Chinchilla。Chinchilla 不仅优于规模大得多的 Gopher，而且较小的模型规模还大幅降低了 inference 成本，并极大地方便了在较小硬件上的下游使用。大语言模型的能源成本会通过其 inference 和 fine-tuning 用途得到摊销。因此，经过更优训练的较小模型所带来的好处，不只限于性能改善这一直接收益。

2. 相关工作

大语言模型。过去几年中，人们提出了多种大语言模型。其中既包括稠密 Transformer 模型 (Brown et al., 2020; Lieber et al., 2021; Rae et al., 2021; Smith et al., 2022; Thoppilan et al., 2022)，也包括 mixture of experts (MoE) 模型 (Du et al., 2021; Fedus

¹例如，已知加速器数量和目标训练时长。

²为简单起见，我们对平滑后的训练损失进行分析；由于我们处在无限数据情形下（训练 token 数量少于整个语料库中的 token 数量），这一损失是测试损失的无偏估计。

et al., 2021; Zoph et al., 2022)。最大的稠密 Transformer 已超过 5000 亿个参数 (Smith et al., 2022)。训练越来越大的模型这一趋势十分明确——迄今为止，扩大语言模型规模一直是推动许多语言建模任务刷新当前最佳水平的原因。不过，大语言模型面临若干挑战，其中包括极其庞大的计算需求（训练与 inference 成本随模型规模增大而上升）(Rae et al., 2021; Thoppilan et al., 2022)，以及获取更多高质量训练数据的需要。事实上，我们在本研究中发现，更大规模的高质量数据集将在语言模型的任何进一步缩放中发挥关键作用。

对缩放行为进行建模。理解语言模型的缩放行为及其迁移特性，一直是近期大型模型发展中的重要环节 (Hernandez et al., 2021; Kaplan et al., 2020)。Kaplan et al. (2020) 率先表明，在多个数量级范围内，模型规模与损失之间存在可预测的关系。这些作者研究了在给定计算预算下应选择最优模型规模来训练。与我们类似，他们也通过训练多种模型来回答这一问题。我们的工作与 Kaplan et al. (2020) 在若干重要方面有所不同。

第一，这些作者对所有模型都采用固定的训练 token 数量和 learning-rate schedule；这使他们无法对这些超参数对损失的影响进行建模。与此相反，我们发现，无论模型规模如何，将 learning-rate schedule 设置为与训练 token 数量大致匹配，都能得到最佳的最终损失——见图 A1。因此，当 learning rate 采用固定到 130B token 的余弦调度时，中间损失估计（对 $D' \ll 130B$ 而言）会高估调度长度与 D' 相匹配的模型的损失。使用这些中间损失，会低估在少于 130B token 的数据上训练模型的有效性，并最终促成这样一个结论：随着计算预算增加，模型规模应比训练数据规模增长得更快。与此相反，我们的分析预测，这两个量应以大致相同的速率缩放。

第二，我们纳入了参数量最高达 16B 的模型，因为我们观察到 FLOP-损失前沿存在轻微曲率（见附录 E）——事实上，我们分析所用的大多数模型参数量超过 5 亿；相比之下，Kaplan et al. (2020) 所用的大多数训练运行规模要小得多，其中许多模型的参数量不足 100M。

最近，Clark et al. (2022) 专门研究了 mixture of experts (MoE) 语言模型的缩放特性，并表明随着模型规模增加，增加专家数量所带来的缩放收益会减弱——他们的方法将损失建模为两个变量的函数：模型规模和专家数量。不过，该分析与 Kaplan et al. (2020) 一样，采用固定的训练 token 数量，因此可能低估了分支机制带来的改进。

估计大型模型的超参数。在选择语言模型及其训练过程时，模型规模和训练 token 数量并非仅有的两个参数。其他重要因素还包括 learning rate、learning-rate schedule、batch size、优化器以及宽深比。在本研究中，我们聚焦于模型规模和训练步数，并依赖现有工作及其给出的实验启发式方法来确定其他必要超参数。Yang et al. (2021) 研究了如何选择训练 autoregressive Transformer 所需的多种参数，其中包括 learning rate 和 batch size。McCandlish et al. (2018) 发现，最优 batch size 与模型规模之间仅存在较弱的依赖关系。Shallue et al. (2018) 和 Zhang et al. (2019) 表明，可以采用比我们更大的 batch size。Levine et al. (2020) 研究了多种标准模型规模下的最优深宽比。我们采用的模型比其建议略浅，因为这能在我们的硬件上带来更好的实际运行时间性能。

改进的模型架构。近来，人们提出了多种有潜力替代传统稠密 Transformer 的方案。例如，通过使用条件计算，1.7 万亿参数的 Switch Transformer (Fedus et al., 2021)、1.2 万亿参数的 GLaM 模型 (Du et al., 2021) 以及其他大型 MoE 模型 (Artetxe et al., 2021; Zoph et al., 2022)，能够在使用相对更少的训练与 inference FLOPs 的情况下提供很大的有效模型规模。不过，对于非常大的模型，路由模型的计算优势似乎会减弱 (Clark et al., 2022)。另一条与之正交的语言模型改进路径，是像 Borgeaud et al. (2021)、Guu et al. (2020) 和 Lewis et al. (2020) 那样，用显式检索机制增强 Transformer。这种方法有效增加了训练期间见到的数据 token 数量（在 Borgeaud et al. (2021) 中约增加到 ~ 10 倍）。这表明，语言模型的性能对训练数据规模的依赖，可能比此前认为的更强。

3. 估计参数量 / 训练 token 数量的最优分配

我们提出三种不同的方法来回答驱动本研究的问题：给定固定的 FLOPs 预算，应如何在模型规模与训练 token 数量之间进行权衡？三种方法都从训练一系列模型开始，同时改变模型规模和训练 token 数量，再使用所得训练曲线拟合一个经验估计器，以估计二者应当如何缩放。与 Clark et al. (2022) 和 Kaplan et al. (2020) 一样，我们假定计算量与模型规模之间存在幂律关系，不过未来工作可能需要把大型模型规模下这种关系的潜在曲率纳入考虑。三种方法得到的预测彼此相近，并表明随着计算量增加，参数量与训练 token 数量应当按相同比例增加³——相应比例见表 2。这与此前关于该主题的研究形成鲜明对比，值得进一步研究。

3.1. 方法 1：固定模型规模，改变训练 token 数量

在第一种方法中，我们针对一组固定规模的模型（参数量从 70M 到超过 10B）改变训练步数，并让每种模型采用 4 种不同的训练序列数量进行训练。根据这些训练运行，我们能够直接提取出在给定训练 FLOPs 数量下所达到的最小损失估计。该方法的训练细节见附录 D。

³FLOPs 的计算方式见附录 F。

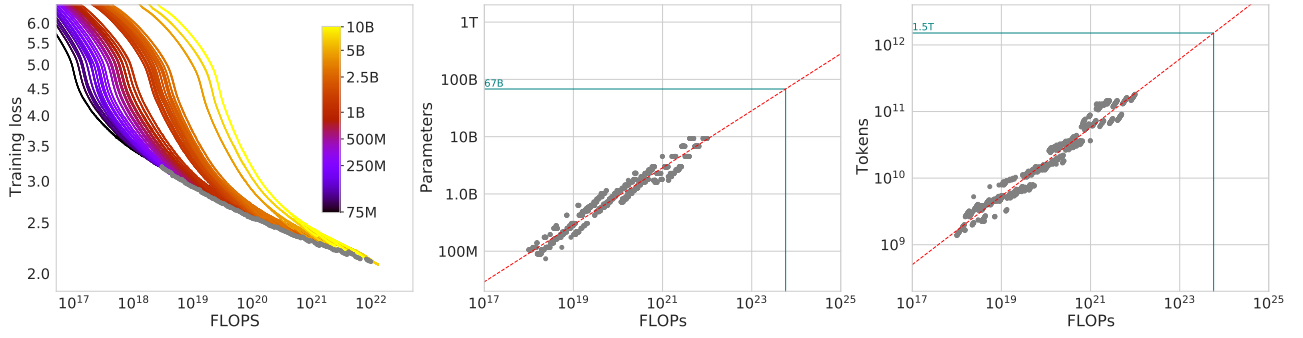


图 2 | 训练曲线包络。左图展示了我们的所有不同训练运行。我们启动了一系列规模从 70M 到 10B 的模型，每种规模采用四种不同的余弦周期长度。我们从这些曲线中提取出每 FLOP 最小损失的包络，并使用这些点估计给定计算预算下的最优模型规模（中图）和最优训练 token 数量（右图）。绿色部分展示了依据训练 Gopher 所用 FLOPs 数量（ 5.76×10^{23} ）推算出的最优模型规模和训练 token 数量。

对于每一种参数数量 N ，我们训练 4 个不同的模型；在以训练 token 数量衡量、跨度相差 $16\times$ 的训练时长内，将 learning rate 衰减 $10\times$ 。随后，我们对每次训练运行的训练损失曲线先进行平滑，再进行插值。由此，对每次训练运行，我们都得到从 FLOP 数量到训练损失的连续映射。接下来，对于每一个 FLOP 数量，我们确定哪次训练运行能达到最低损失。利用这些插值函数，我们便得到从任意 FLOP 数量 C 到最高效的模型规模 N 与训练 token 数量 D 选择之间的映射，其中 $\text{FLOPs}(N, D) = C$ 。⁴ 对 1500 个按对数间隔取值的 FLOP 数，我们找出在所有模型中达到最低损失的模型规模，以及所需的训练 token 数量。最后，我们拟合幂律，以估计任意给定计算量下的最优模型规模和训练 token 数量（见图 2 的中图和右图），得到关系 $N_{\text{opt}} \propto C^a$ 和 $D_{\text{opt}} \propto C^b$ 。我们发现 $a = 0.50$ 、 $b = 0.50$ ，如表 2 所总结。在第 D.4 节中，我们在 10^{21} FLOPs 下，使用我们的分析与 Kaplan et al. (2020) 的分析分别建议的模型规模进行正面对比——采用我们预测的模型规模具有明显优势。

3.2. 方法 2：等 FLOP 剖面

在第二种方法中，我们改变模型规模⁵，并针对一组固定的 9 种不同训练 FLOP 数量⁶（从 6×10^{18} 到 3×10^{21} FLOPs），考察每个点的最终训练损失⁷。这不同于方法 1；后者考察的是整个训练运行过程中沿途的 (N, D, L) 点。这样我们便能直接回答：对于给定的 FLOP 预算，最优参数数量是多少？

对于每个 FLOP 预算，我们在图 3（左）中将最终损失（经平滑后）相对于参数数量作图。在所有情形下，我们都确保训练了足够多样的模型规模，使损失中出现清晰的最小值。我们对每条等 FLOP 曲线拟合一个抛物线，以直接估计在哪个模型规模下达到最小损失（图 3（左））。与上一种方法一样，随后我们在 FLOPs 与损失最优的模型规模及训练 token 数量之间拟合幂律，如图 3（中、右）所示。我们再次拟合形如 $N_{\text{opt}} \propto C^a$ 和 $D_{\text{opt}} \propto C^b$ 的指数关系，得到 $a = 0.49$ 、 $b = 0.51$ ，如表 2 所总结。

⁴请注意，所有选中的点都位于训练最后 15% 的区间内。这表明，当用 D 个 token 训练一个模型时，应选择一个在大约 D 个 token 期间衰减 $10\times$ 的余弦周期长度——更多细节见附录 B。

⁵在方法 2 中，模型规模最高达 16B；而方法 1 只使用了最高 10B 的模型。

⁶训练 token 数量由模型规模和训练 FLOPs 决定。

⁷我们将余弦调度长度设置为与 token 数量相匹配；根据附录 B 中给出的分析，这是最优设置。

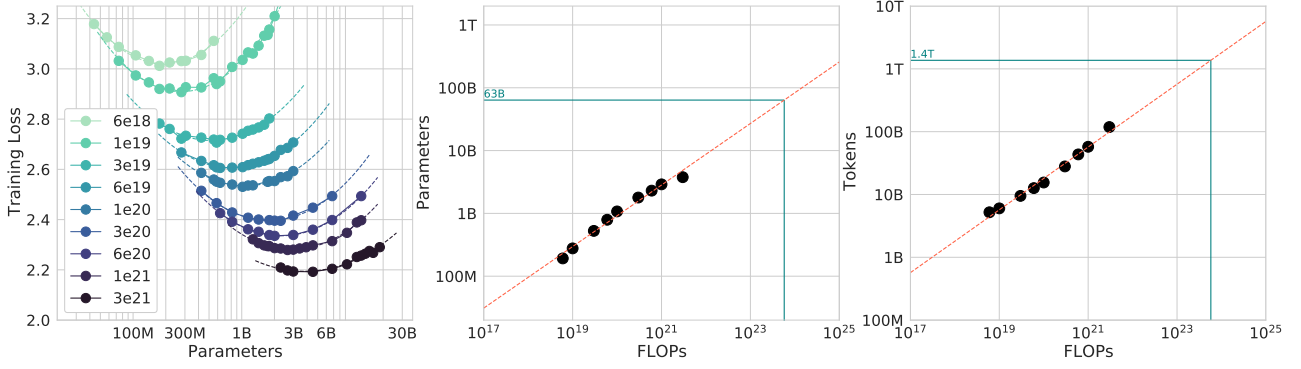


图 3 | 等 FLOP 曲线。对于不同的模型规模，我们选择训练 token 数量，使最终 FLOPs 保持为常数。余弦周期长度被设置为与目标 FLOP 数量相匹配。我们观察到损失存在清晰的谷底，这意味着对于给定 FLOP 预算，存在一个最优的待训练模型（左图）。利用这些谷底的位置，我们推算出更大模型的最优模型规模和 token 数量（中图和右图）。绿色部分展示了使用 Gopher 的计算预算训练一个最优模型时，所估计的参数数量和 token 数量。

3.3. 方法 3：拟合参数化损失函数

最后，我们把方法 1 和方法 2 的实验所得全部最终损失，建模为模型参数数量与已见 token 数量的参数化函数。按照经典的风险分解（见第 D.2 节），我们提出如下函数形式：

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}. \quad (2)$$

第一项刻画数据分布上理想生成过程的损失，并且应当对应自然文本的熵。第二项刻画这样一个事实：一个经过完美训练、具有 N 个参数的 Transformer，表现仍不及理想生成过程。最后一项刻画这样一个事实：Transformer 尚未训练至收敛，因为我们只在数据集分布的一个样本上执行了有限次数的优化步骤。

模型拟合。为估计 (A, B, E, α, β) ，我们使用 L-BFGS 算法 (Nocedal, 1980)，最小化预测对数损失与观测对数损失之间的 Huber 损失 (Huber, 1964)：

$$\min_{A, B, E, \alpha, \beta} \sum_{\text{Runs } i} \text{Huber}_\delta \left(\log \hat{L}(N_i, D_i) - \log L_i \right). \quad (3)$$

为处理可能存在的局部极小值，我们从一组网格化初始化中选择最佳拟合。Huber 损失 ($\delta = 10^{-3}$) 对离群值具有稳健性；我们发现，这对于在留出数据点上取得良好预测性能十分重要。第 D.2 节详述了拟合过程和损失分解。

有效前沿。我们可以在约束 $\text{FLOPs}(N, D) \approx 6ND$ (Kaplan et al., 2020) 下最小化参数化损失 $\hat{L}$ ，从而近似函数 N_{opt} 和 D_{opt} 。所得 N_{opt} 和 D_{opt} 在公式 (3) 中依赖模型规模和数据的两项之间取得平衡。按照构造，它们具有幂律形式：

$$N_{\text{opt}}(C) = G \left(\frac{C}{6} \right)^a, \quad D_{\text{opt}}(C) = G^{-1} \left(\frac{C}{6} \right)^b, \quad (4)$$

其中 $G = \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha + \beta}}, \quad a = \frac{\beta}{\alpha + \beta}, \quad b = \frac{\alpha}{\alpha + \beta}.$

我们在图 4（左）中展示了拟合函数 $\hat{L}$ 的等高线，并以蓝色展示其闭式计算高效前沿。由这种方法，我们得到 $a = 0.46$ 、 $b = 0.54$ ，如表 2 所总结。

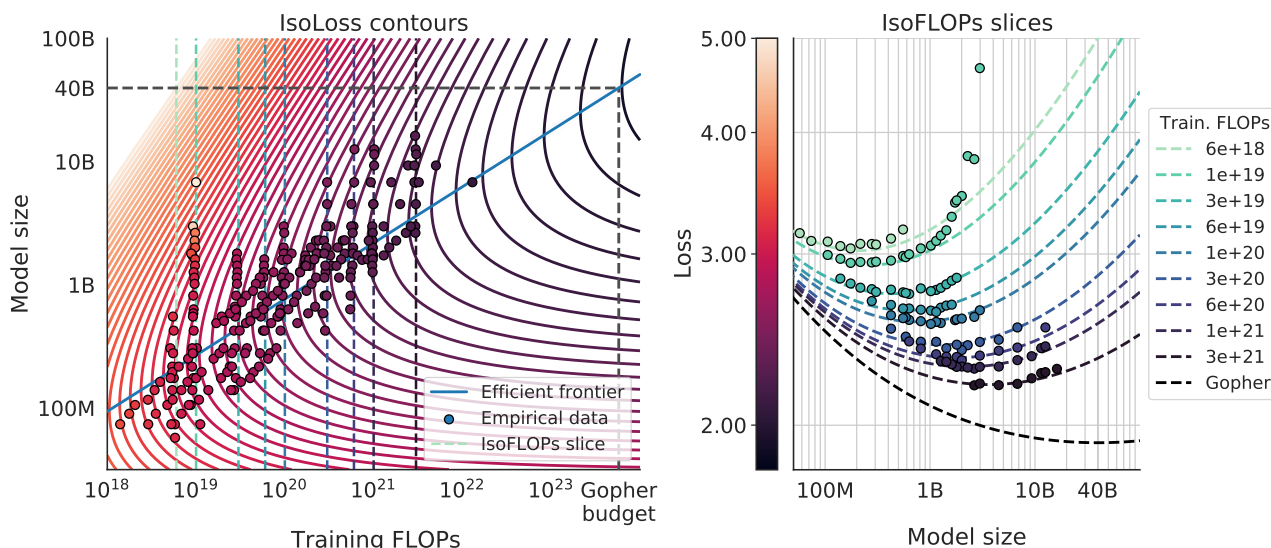


图 4 | 参数化拟合。我们对损失 $\hat{L}(N, D)$ 进行参数化建模，并展示等高线（左图）和等 FLOP 切片（右图）。对每一个等 FLOP 切片，我们都在左图中加入一条对应的虚线。在左图中，我们用蓝色展示有效前沿，它在双对数空间中是一条直线。具体而言，该曲线穿过每一条等损失等高线中 FLOPs 最少的点。我们推算，在 Gopher 的 FLOP 预算下，最优模型规模为 40B 参数。

3.4. 最优模型缩放

我们发现，尽管三种方法采用了不同的拟合方法和不同的已训练模型，但它们对于参数和 token 如何随 FLOPs 进行最优缩放，给出了相近的预测（见表 2）。

方法	系数 a ，其中 $N_{\text{opt}} \propto C^a$	系数 b ，其中 $D_{\text{opt}} \propto C^b$
1. 训练曲线上的最小值	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. 等 FLOP 剖面	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. 损失的参数化建模	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

表 2 | 训练计算量增加时参数和数据的估计缩放。所列数值是关系 $N_{\text{opt}} \propto C^a$ 和 $D_{\text{opt}} \propto C^b$ 中的指数 a 与 b 。我们的分析表明，随着计算量增加，参数和数据近乎等比例缩放；这与此前关于大型模型缩放的研究形成鲜明对比。第 10 百分位数和第 90 百分位数通过对数据进行自助法抽样来估计（每次抽取数据集的 80%，共抽样 100 次），并列于括号中。

三种方法都表明，随着计算预算增加，模型规模和训练数据量应以近似相等的比例增加。如图 1 和图 A3 所示，第一种和第二种方法对最优模型规模给出了非常相近的预测。第三种方法则预测，在较大的计算预算下，规模更小的模型反而最优。

我们注意到，训练 FLOPs 较低 ($C \leq 1e21$) 的观测点 (L, N, D) ，其残差 $\|L - \hat{L}(N, D)\|_2^2$ 大于计算预算更高的点。拟合模型赋予 FLOPs 更多的点更高权重——借助 Huber 损失，它自动将低计算预算的点视为离群值。由于在前沿 $C \rightarrow N_{\text{opt}}$ 中经验观察到负曲率（见附录 E），其结果是预测出的 N_{opt} 低于另外两种方法。

参数量	FLOPs	FLOPs (以 Gopher 为单位)	token
4 亿	1.92e+19	1/29,968	80 亿
10 亿	1.21e+20	1/4,761	202 亿
100 亿	1.23e+22	1/46	2051 亿
670 亿	5.76e+23	1	1.5 万亿
1750 亿	3.85e+24	6.7	3.7 万亿
2800 亿	9.90e+24	17.2	5.9 万亿
5200 亿	3.43e+25	59.5	11.0 万亿
1 万亿	1.27e+26	221.3	21.2 万亿
10 万亿	1.30e+28	22515.9	216.2 万亿

表 3 | 不同模型规模下估计的最优训练 FLOPs 和训练 token。对于不同的模型规模，我们给出方法 1 的推算结果，即训练计算最优模型需要多少 FLOPs 和训练 token。方法 2 和方法 3 的估计结果相近（见第 D.3 节）。

表 3 给出了为确保给定规模的模型处于计算最优前沿而估计所需的 FLOPs 和 token 数量。

我们的发现表明，如图 1 所示，当前一代大语言模型相对于各自的计算预算而言规模明显过大。例如，我们发现，1750 亿参数的模型应当使用 4.41×10^{24} FLOPs 的计算预算，并在超过 4.2 万亿 token 上进行训练。类似 Gopher 的 2800 亿参数模型，在约 10^{25} FLOPs 的计算预算下才是最优的待训练模型，并且应在 6.8 万亿 token 上训练。除非拥有 10^{26} FLOPs 的计算预算（超过训练 Gopher 所用计算量的 250 $\times$ ），否则 1 万亿参数模型不太可能是最优的待训练模型。

此外，推算所需的训练数据量远远超过当前训练大型模型时所用的数据量，这凸显了数据集收集的重要性；其重要程度不亚于那些使模型规模得以扩大所需的工程改进。虽然跨越多个数量级进行外推存在显著不确定性，但我们的分析清楚表明，对于许多当前 LLM 的训练计算预算，若要得到性能最佳的模型，本应在更多 token 上训练更小的模型。

在附录 C 中，我们还在另外两个数据集上复现了等 FLOP 分析：C4 (Raffel et al., 2020a) 和 GitHub 代码 (Rae et al., 2021)。在两种情况下，我们都得出了相似结论：模型规模与训练 token 数量应当按相同比例缩放。

4. Chinchilla

根据第 3 节的分析，在 Gopher 的计算预算下，最优模型规模介于 400 亿到 700 亿参数之间。出于数据集和计算效率两方面的考虑，我们通过训练该区间较大一端的模型——一个在 1.4T token 上训练的 70B 参数模型——来检验这一假设。在本节中，我们将这个称为 Chinchilla 的模型与 Gopher 及其他 LLM 进行比较。Chinchilla 和 Gopher 使用相同数量的 FLOPs 进行训练，但二者在模型规模和训练 token 数量上有所不同。

虽然 pre-training 大语言模型的计算成本相当可观，但下游 fine-tuning 和 inference 同样占用了大量计算资源 (Rae et al., 2021)。由于 Chinchilla 比 Gopher 小 4 $\times$ ，它的内存占用和 inference 成本也都更低。

4.1. 模型与训练细节

训练 Chinchilla 所用的全套超参数见表 4。除下列差异外，Chinchilla 采用与 Gopher 相同的模型架构和训练设置。

- 我们在 MassiveText（与 Gopher 相同的数据集）上训练 Chinchilla，但使用略有不同的子集分布（见表 A1），以适应增加的训练 token 数量。
- Chinchilla 使用 AdamW (Loshchilov and Hutter, 2019)，而非 Adam (Kingma and Ba, 2014)，因为前者能改善语言建模损失和 fine-tuning 后的下游任务性能。⁸
- 我们使用略作修改的 SentencePiece (Kudo and Richardson, 2018) tokenizer 训练 Chinchilla，该 tokenizer 不应用 NFKC 规范化。其词表非常相似——94.15% 的 token 与训练 Gopher 所用的 token 相同。我们发现，这尤其有助于数学和化学等内容的表示。
- 前向传播和反向传播使用 bfloat16 计算，但我们在分布式优化器状态中保存一份 float32 权重副本 (Rajbhandari et al., 2020)。更多细节参见 Rae et al. (2021) 的“经验教训” (Lessons Learned)。

模型	层数	注意力头数量	键/值大小	d_{model}	最大 learning rate	batch size
Gopher 280B	80	128	128	16,384	4×10^{-5}	3M $\rightarrow$ 6M
Chinchilla 70B	80	64	128	8,192	1×10^{-4}	1.5M $\rightarrow$ 3M

表 4 | Chinchilla 架构细节。我们列出了层数、键 / 值大小、瓶颈激活大小 d_{model} 、最大 learning rate，以及训练 batch size (token 数量)。前馈层大小始终设为 $4 \times d_{\text{model}}$ 。请注意，对 Chinchilla 和 Gopher，我们都在训练进行到一半时将 batch size 加倍。

在附录 G 中，我们展示了 Chinchilla 与 Gopher 之间各种优化器相关改动的影响。本分析中的所有模型都使用 JAX (Bradbury et al., 2018) 和 Haiku (Hennigan et al., 2020) 在 TPUv3/TPUv4 (Jouppi et al., 2017) 上训练。表 A8 中包含 Chinchilla 的模型卡 (Mitchell et al., 2019)。

4.2. 结果

我们对 Chinchilla 进行了广泛评估，并将其与多种大语言模型进行比较。我们在 Rae et al. (2021) 所列任务的一个大型子集上进行评估，任务见表 5。由于本研究关注的是最优模型缩放，我们纳入了一个具有代表性的大型子集，并新增了少量评估，以便更好地与其他现有大型模型比较。所有任务的评估细节均与 Rae et al. (2021) 中的描述相同。

⁸有趣的是，使用 AdamW 训练的模型，要到余弦周期约 80% 处，其训练性能才超过使用 Adam 训练的模型；不过，其最终性能明显更好——见图 A7。

类别	任务数	示例
语言建模	20	WikiText-103、The Pile: PG-19、arXiv、FreeLaw、……
阅读理解	3	RACE-m、RACE-h、LAMBADA
问答	3	Natural Questions、TriviaQA、TruthfulQA
常识	5	HellaSwag、Winogrande、PIQA、SIQA、BoolQ
MMLU	57	高中化学 (High School Chemistry)、天文学 (Astronomy)、临床知识 (Clinical Knowledge)、……
BIG-bench	62	因果判断 (Causal Judgement)、认识论推理 (Epistemic Reasoning)、时间序列 (Temporal Sequences)、……

表 5 | 全部评估任务。我们在一组语言建模任务和下游任务上评估 Chinchilla。为便于直接比较，我们评估的任务与 Rae et al. (2021) 中的任务大体相同。

4.2.1. 语言建模

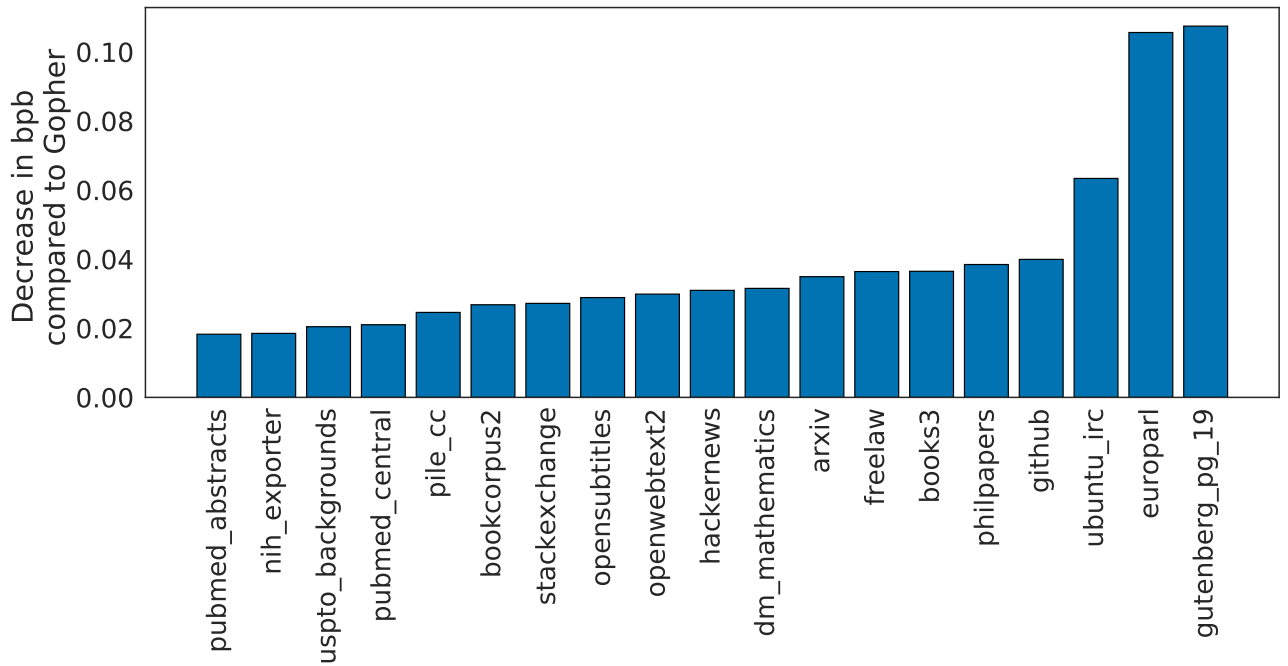


图 5 | Pile 评估。对于 The Pile (Gao et al., 2020) 中的不同评估集，我们展示了 Chinchilla 相较 Gopher 在每字节比特数 (bpb) 上的改善（下降量）。在所有子集上，Chinchilla 都优于 Gopher。

如图 5 所示，在 The Pile (Gao et al., 2020) 的所有评估子集上，Chinchilla 都显著优于 Gopher。与 Jurassic-1 (178B) (Lieber et al., 2021) 相比，除两个子集 `dm_mathematics` 和 `ubuntu_irc` 外，Chinchilla 在其余所有子集上的表现都更好；每字节比特数的原始比较见表 A5。在 Wikitext103 (Merity et al., 2017) 上，Chinchilla 的困惑度达到 7.16，而 Gopher 为 7.75。在这些语言建模基准上比较 Chinchilla 与 Gopher 时需要谨慎，因为 Chinchilla 的训练数据比 Gopher 多 4×，因此训练集 / 测试集泄漏可能会人为提高结果。因此，我们更加重视其他不太需要担心泄漏的任务，例如 MMLU (Hendrycks et al., 2020) 和 BIG-bench (BIG-bench collaboration, 2021)，以及多种闭卷问答和常识分析。

4.2.2. MMLU

对象	准确率
随机	25.0%
人类评分者平均水平	34.5%
GPT-3 five-shot	43.9%
Gopher five-shot	60.0%
Chinchilla five-shot	67.6%
人类专家平均表现	89.8%
2022 年 6 月预测	57.1%
2023 年 6 月预测	63.4%

表 6 | 大规模多任务语言理解 (MMLU)。我们报告了 57 项任务上的平均 five-shot 准确率；模型与人类准确率的比较取自 Hendrycks et al. (2020)。我们还列出了 Steinhardt (2021) 中 73 位参与竞争的人类预测者，对 2022 年 6 月 / 2023 年 6 月当前最佳准确率所作预测的平均值。

大规模多任务语言理解 (MMLU) 基准 (Hendrycks et al., 2020) 包含一系列学术科目的考试式问题。在表 6 中，我们报告了 Chinchilla 在 MMLU 上的平均 five-shot 表现 (完整结果明细见表 A6)。在该基准上，尽管 Chinchilla 小得多，但它仍显著优于 Gopher，平均准确率为 67.6% (比 Gopher 提高 7.6%)。值得注意的是，Chinchilla 甚至超过了专家对 2023 年 6 月所预测的 63.4% 准确率 (见表 6) (Steinhardt, 2021)。此外，Chinchilla 在 4 个不同的单项任务上达到超过 90% 的准确率——`high_school_gov_and_politics` (高中政府与政治)、`international_law` (国际法)、`sociology` (社会学) 和 `us_foreign_policy` (美国外交政策)。据我们所知，此前没有其他模型在其中任一子任务上达到超过 90% 的准确率。

图 6 按任务展示了与 Gopher 的比较。总体而言，我们发现 Chinchilla 在绝大多数任务上都提升了性能。在四项任务 (`college_mathematics`、`econometrics`、`moral_scenarios` 和 `formal_logic`) 上，Chinchilla 的表现不及 Gopher；另有两项任务的性能没有变化。

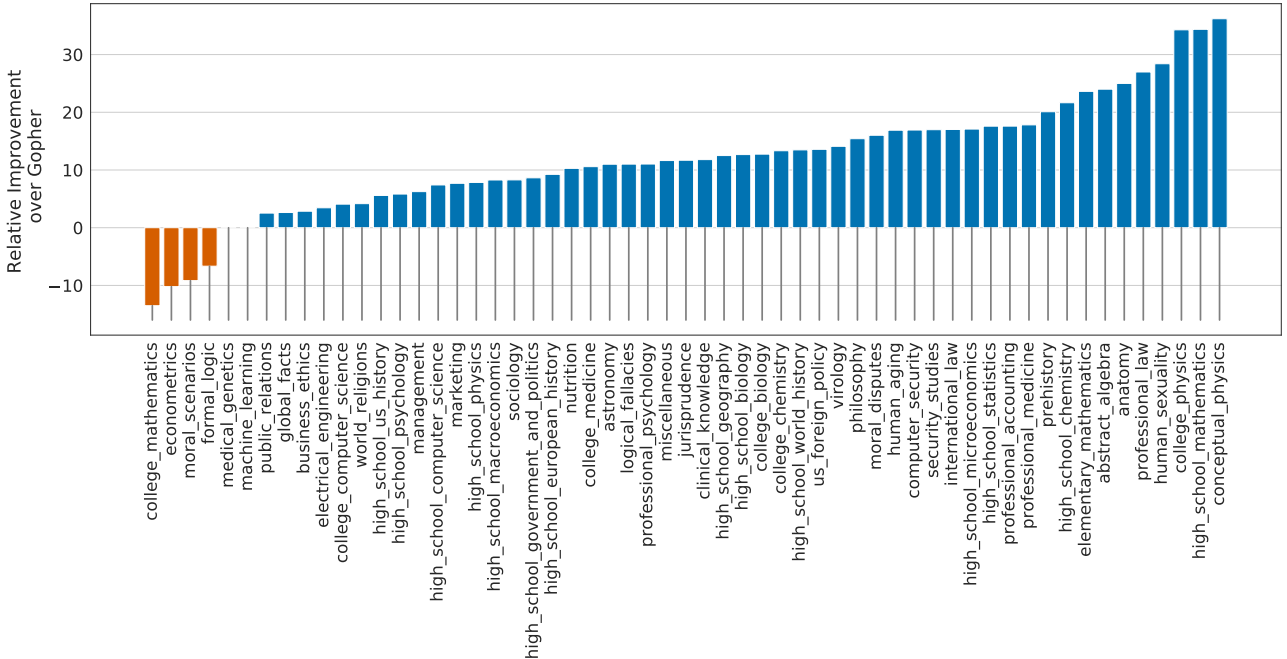


图 6 | MMLU 结果与 Gopher 的比较。我们发现，Chinchilla 平均比 Gopher 高 7.6% (见表 6)；此外，在 57 项单独任务中，Chinchilla 在 51/57 项上表现更好，在 2/57 项上表现相同，仅在 4/57 项上表现更差。

4.2.3. 阅读理解

在末词预测数据集 LAMBADA (Paperno et al., 2016) 上，Chinchilla 的准确率达到 77.4%，相比之下，Gopher 为 74.5%，MT-NLG 530B 为 76.6% (见表 7)。在 RACE-h 和 RACE-m (Lai et al., 2017) 上，Chinchilla 大幅优于 Gopher，在两种情况下准确率均提高 10% 以上——见表 7。

任务与设置	Chinchilla	Gopher	GPT-3	MT-NLG 530B
LAMBADA zero-shot	77.4	74.5	76.2	76.6
RACE-m few-shot	86.8	75.1	58.1	-
RACE-h few-shot	82.3	71.6	46.8	47.9

表 7 | 阅读理解。在 RACE-h 和 RACE-m (Lai et al., 2017) 上，Chinchilla 的表现相较 Gopher 有显著提升。请注意，在 RACE-h/m 上，GPT-3 和 MT-NLG 530B 使用的提示格式与我们不同，因此它们的结果不能与 Gopher 和 Chinchilla 比较。在 LAMBADA (Paperno et al., 2016) 上，Chinchilla 同时优于 Gopher 和 MT-NLG 530B。

4.2.4. BIG-bench

我们在 Rae et al. (2021) 所报告的同一组 BIG-bench 任务 (BIG-bench collaboration, 2021) 上分析了 Chinchilla。与我们在 MMLU 上的观察类似，Chinchilla 在绝大多数任务上都优于 Gopher (见图 7)。我们发现，Chinchilla 将平均性能提升了 10.7%，准确率达到 65.1%，而 Gopher 为 54.4%。在所考察的 62 项任务中，Chinchilla 仅在四项上不及 Gopher——`crash_blossom`、`dark_humor_detection`、`mathematical_induction` 和 `logical_args`。Chinchilla 的完整准确率结果见表 A7。

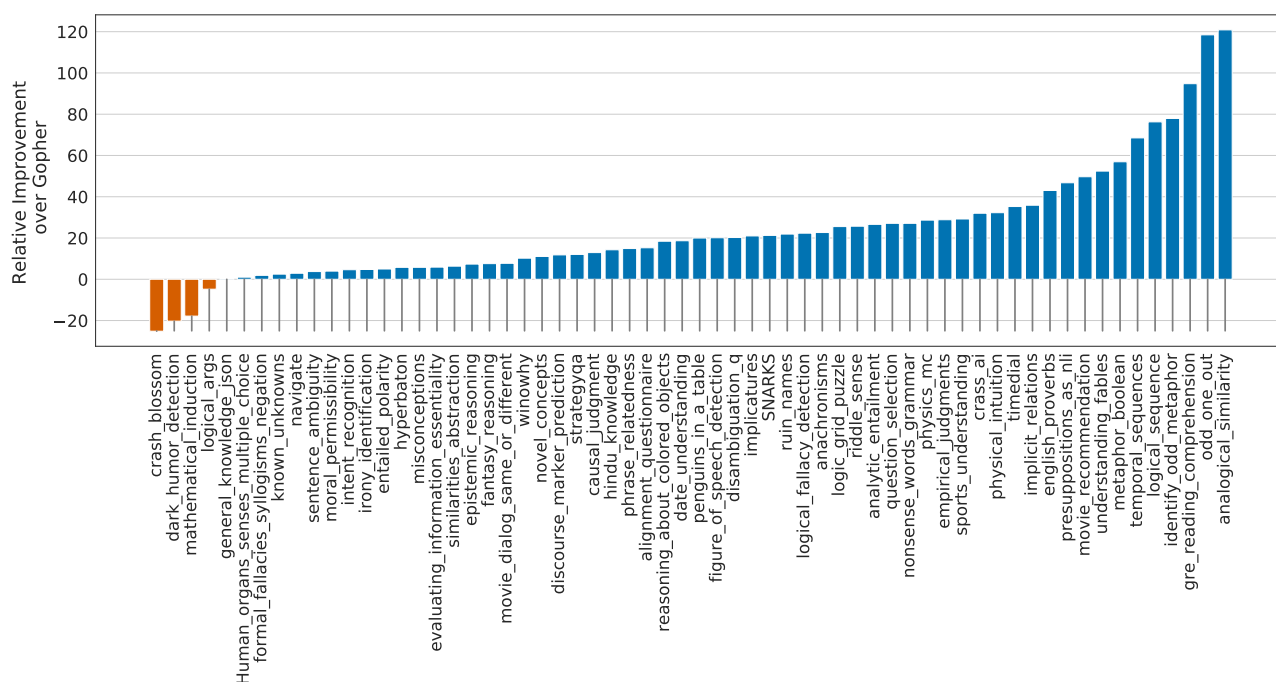


图 7 | 与 Gopher 相比的 BIG-bench 结果。在所考虑的 BIG-bench 任务中，除四项外，Chinchilla 在其余所有任务上都优于 Gopher。完整结果见表 A7。

4.2.5. 常识

基准	Chinchilla	Gopher	GPT-3	MT-NLG 530B	监督式 SOTA (当前最佳)
HellaSWAG	80.8%	79.2%	78.9%	80.2%	93.9%
PIQA	81.8%	81.8%	81.0%	82.0%	90.1%
Winogrande	74.9%	70.1%	70.2%	73.0%	91.3%
SIQA	51.3%	50.6%	-	-	83.2%
BoolQ	83.7%	79.3%	60.5%	78.2%	91.4%

表 8 | 常识基准上的 zero-shot 比较。我们比较了 Chinchilla、Gopher 和 MT-NLG 530B 在多项常识基准上的表现。可以看到，在所有任务上，Chinchilla 都达到或优于 Gopher 和 GPT-3。除一项任务外，Chinchilla 还优于规模大得多的 MT-NLG 530B 模型。

我们在多种常识基准上评估 Chinchilla：PIQA (Bisk et al., 2020)、SIQA (Sap et al., 2019)、Winogrande (Sakaguchi et al., 2020)、HellaSwag (Zellers et al., 2019) 和 BoolQ (Clark et al., 2019)。我们发现，Chinchilla 在所有任务上都优于 Gopher 和 GPT-3，并且除一项任务外，在其余所有任务上都优于 MT-NLG 530B——见表 8。

在 TruthfulQA (Lin et al., 2021) 上，Chinchilla 在 0-shot (zero-shot)、5-shot (five-shot) 和 10-shot (ten-shot) 设置下的准确率分别达到 43.6%、58.5% 和 66.7%。相比之下，Gopher 在 0-shot 下仅达到 29.5%，在 10-shot 下仅达到 43.7%。这与 Lin et al. (2021) 的发现形成鲜明对比：Chinchilla 所取得的大幅提升 (0-shot 准确率提高 14.1%) 表明，仅通过更好地建模 pre-training 数据，也能在该基准上带来显著改善。

4.2.6. 闭卷问答

闭卷问答基准的结果见表 9。在 Natural Questions 数据集 (Kwiatkowski et al., 2019) 上，Chinchilla 取得了新的闭卷问答当前最佳准确率：five-shot (5-shot) 为 31.5%，64 样本 (64-shot) 为 35.5%；相比之下，Gopher 分别为 21% 和 28%。在 TriviaQA (Joshi et al., 2017) 上，我们同时给出过滤后集合（此前用于检索和开卷工作）与未过滤集合（此前用于大语言模型评估）的结果。在这两种情况下，Chinchilla 都显著优于 Gopher。在过滤后版本上，Chinchilla 仅落后于开卷当前最佳结果 (Izacard and Grave, 2020) 7.9%。在未过滤集合上，Chinchilla 优于 GPT-3，见表 9。

数据集与划分	方法	Chinchilla	Gopher	GPT-3	SOTA (开卷)
Natural Questions (dev)	zero-shot (0-shot)	16.6%	10.1%	14.6%	54.4%
Natural Questions (dev)	five-shot (5-shot)	31.5%	24.5%	-	54.4%
Natural Questions (dev)	64 样本 (64-shot)	35.5%	28.2%	29.9%	54.4%
TriviaQA (未过滤, test)	zero-shot (0-shot)	67.0%	52.8%	64.3%	-
TriviaQA (未过滤, test)	five-shot (5-shot)	73.2%	63.6%	-	-
TriviaQA (未过滤, test)	64 样本 (64-shot)	72.3%	61.3%	71.2%	-
TriviaQA (过滤后, dev)	zero-shot (0-shot)	55.4%	43.5%	-	72.5%
TriviaQA (过滤后, dev)	five-shot (5-shot)	64.1%	57.0%	-	72.5%
TriviaQA (过滤后, dev)	64 样本 (64-shot)	64.6%	57.2%	-	72.5%

表 9 | 闭卷问答。对于 Natural Questions (Kwiatkowski et al., 2019) 和 TriviaQA (Joshi et al., 2017), Chinchilla 在所有情况下都优于 Gopher。在 Natural Questions 上, Chinchilla 优于 GPT-3。在 TriviaQA 上, 我们给出两个不同评估集合的结果, 以便分别与 GPT-3 以及开卷 SOTA (FiD + Distillation (Izacard and Grave, 2020)) 进行比较。

4.2.7. 性别偏见与毒性

大语言模型可能带来一些风险, 例如输出冒犯性语言、传播社会偏见, 以及泄露私人信息 (Bender et al., 2021; Weidinger et al., 2021)。我们预计 Chinchilla 所带来的风险与 Gopher 相似, 因为 Chinchilla 使用相同的数据训练, 尽管其相对权重略有不同, 而且它采用了相似的架构。

这里, 我们考察性别偏见 (尤其是性别与职业偏见) 以及毒性语言的生成。我们选择了几种常见评估来凸显潜在问题, 但要强调的是, 我们的评估并不全面; 在理解、评估和缓解大语言模型风险方面, 仍有大量工作要做。**性别偏见。**

正如 Rae et al. (2021) 所讨论的, 大语言模型会反映其训练数据集中有关不同群体 (例如性别群体) 的当代和历史话语, 我们预计 Chinchilla 也是如此。这里, 我们在 zero-shot 设置下使用 Winogender 数据集 (Rudinger et al., 2018), 检验潜在的性别与职业偏见是否会在共指消解中表现为不公平的结果。Winogender 检验模型能否正确判断一个代词指向哪个职业词。无偏模型无论代词的性别为何, 都应正确预测该代词所指向的词。我们采用与 Rae et al. (2021) 相同的设置 (第 H.3 节有进一步说明)。

如表 10 所示, 在所有群体中, Chinchilla 正确消解代词的频率都高于 Gopher。有意思的是, 男性代词的性能提升 (提升 3.2%) 明显小于女性或中性代词 (分别提升 8.3% 和 9.2%)。我们还考虑了 gotcha 样例, 在这些样例中, 正确的代词消解与性别刻板印象相矛盾 (刻板印象根据劳工统计数据确定)。我们再次看到, Chinchilla 对代词的消解比 Gopher 更准确。将样例按男性/女性以及 gotcha/not gotcha 拆分后, 最大的提升出现在女性 gotcha 样例上 (提升 10%)。因此, 尽管 Chinchilla 相比 Gopher 能在更多共指样例中一致地克服性别刻板印象, 但不同代词的提升速度并不相同, 这表明使用更为计算最优的模型所带来的改进可能并不均匀。

群体	Chinchilla	Gopher
全部	78.3%	71.4%
男性	71.2%	68.0%
女性	79.6%	71.3%
中性	84.2%	75.0%

样例类别	Chinchilla	Gopher
男性 gotcha	62.5%	59.2%
男性 not gotcha	80.0%	76.7%
女性 gotcha	76.7%	66.7%
女性 not gotcha	82.5%	75.8%

表 10 | Winogender 结果。左: Chinchilla 在代词消解上始终优于 Gopher。右: Chinchilla 在与性别刻板印象相矛盾的样例 (gotcha 样例) 上表现更好。然而, 各群体之间的性能差异表明 Chinchilla 存在偏见。

样本毒性。语言模型能够生成毒性语言, 包括侮辱、仇恨言论、亵渎用语和威胁 (Gehman et al., 2020; Rae et al., 2021)。尽管“毒性”是一个宽泛术语, 而且在语言模型中评估毒性面临诸多挑战 (Welbl et al., 2021; Xu et al., 2021), 自动分类器的分数仍可用于指示语言模型所生成有害文本的程度。Rae et al. (2021) 发现, 通过增加模型参数数量来改善语言建模损失, 对毒性文本生成 (无提示) 的影响微乎其微; 这里我们分析通过更为计算最优的训练得到更低的语言模型损失时, 情况是否仍然如此。

与 Rae et al. (2021) 的方案类似, 我们从 Chinchilla 生成 25,000 个无提示样本, 并将这些样本的 PerspectiveAPI 毒性分数分布与 Gopher 生成样本的分布进行比较。若干汇总统计量表明不存在重大差异: Gopher 的平均 (中位数) 毒性分数为 0.081 (0.064), Chinchilla 则为 0.087 (0.066); 第 95 百分位的分数, Gopher 为 0.230, Chinchilla 为 0.238。也就是说, 绝大多数生成样本都被归类为无毒, 两个模型之间的差异可以忽略不计。与先前的发现 (Rae et al., 2021) 一致, 这表明无条件文本生成中的毒性水平在很大程度上独立于模型质量 (以语言建模损失衡量); 换言之, 更好地建模训练数据集并不一定会使模型更具毒性。

5. 讨论与结论

迄今为止，大语言模型训练的发展趋势一直是增大模型规模，却往往不增加训练 token 的数量。最大的稠密 Transformer MT-NLG 530B，如今已是仅仅两年前拥有 170B (1700 亿) 参数的 GPT-3 的 3 倍以上。然而，这个模型以及现有大多数大模型，训练所用的 token 数量都相近，约为 300B (3000 亿)。尽管训练这些巨型模型的愿望推动了大量工程创新，但我们假设，这场训练越来越大模型的竞赛，正在导致模型的表现显著低于使用相同计算预算所能达到的水平。

基于 400 多次训练运行的结果，我们提出了三种预测方法，用于以最优方式设定模型规模和训练时长。三种方法都预测 Gopher 的规模明显过大，并估计在相同计算预算下，用更多数据训练一个较小的模型会表现得更好。我们通过训练一个拥有 70B 参数的模型 Chinchilla 来直接检验这一假设，并表明它在几乎所有测得的评估任务上都优于 Gopher，甚至优于规模更大的模型。

虽然我们的方法让我们能够预测在获得额外计算量时应如何缩放大模型，但它仍有若干局限。由于训练大模型的成本很高，我们在大规模上只有两次可比较的训练运行 (Chinchilla 和 Gopher)，在中等规模上也没有额外测试。此外，我们假设，计算高效前沿可以用计算预算、模型规模和训练 token 数量之间的幂律关系描述。然而，在高计算预算下，我们观察到 $\log(N_{\text{opt}})$ 存在一定凹性 (见附录 E)。这表明，我们可能仍在高估大模型的最优规模。最后，用于分析的所有训练运行都只在少于一个数据轮次上完成训练；未来工作可以考虑多轮次情形。尽管存在这些局限，Chinchilla 与 Gopher 的比较验证了我们的性能预测，而这些预测使我们能够在相同计算预算下训练出一个更好 (也更轻量) 的模型。

尽管最近已有大量工作使越来越大的模型得以训练，但我们的分析表明，需要更加重视数据集缩放。推测而言，我们预计，只有当数据质量较高时，向越来越大的数据集缩放才会带来益处。这要求以负责任的方式收集更大的数据集，并高度重视数据集质量。对于更大的数据集，需要格外谨慎，以确保无论在语言建模损失还是在下游任务中，都正确考虑训练集与测试集的重叠。最后，使用数万亿 token 进行训练会引入许多伦理与隐私问题。从网络抓取的大型数据集会包含毒性语言、偏见和私人信息。使用更大的数据集时，此类信息的数量 (即便不是出现频率) 会增加，这使得对数据集进行内部审视变得更加重要。Chinchilla 的确存在偏见和毒性，但有意思的是，它受到的影响似乎小于 Gopher。更好地理解大语言模型的性能与毒性如何相互作用，是一个重要的未来研究问题。

尽管我们将这套方法应用于 autoregressive 语言模型的训练，但我们预计，在其他模态中，模型规模与数据量之间也存在类似的权衡。由于训练大模型非常昂贵，在训练开始之前选择最优模型规模和训练步数至关重要。我们提出的方法很容易在新的设置中复现。

6. 致谢

感谢 Jean-baptiste Alayrac、Kareem Ayoub、Chris Dyer、Nando de Freitas、Demis Hassabis、Geoffrey Irving、Koray Kavukcuoglu、Nate Kushman 和 Angeliki Lazaridou 对本文稿提出的有益意见。感谢 Andy Brock、Irina Higgins、Michela Paganini、Francis Song 以及 DeepMind 的其他同事参与有益讨论。我们还非常感谢 JAX 和 XLA 团队提供的支持与帮助。

参考文献

- M. Artetxe, S. Bhosale, N. Goyal, T. Mihaylov, M. Ott, S. Shleifer, X. V. Lin, J. Du, S. Iyer, R. Pasunuru, G. Anantharaman, X. Li, S. Chen, H. Akin, M. Baines, L. Martin, X. Zhou, P. S. Koura, B. O'Horo, J. Wang, L. Zettlemoyer, M. Diab, Z. Kozareva, and V. Stoyanov. Efficient Large Scale Language Modeling with Mixtures of Experts. arXiv:2112.10684, 2021.
- E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, pages 610-623, 2021.
- BIG-bench collaboration. Beyond the imitation game: Measuring and extrapolating the capabilities of language models. In preparation, 2021. URL <https://github.com/google/BIG-bench/>.
- Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al. PIQA: Reasoning about physical commonsense in natural language. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 34, pages 7432-7439, 2020.
- S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre. Improving language models by retrieving from trillions of tokens. arXiv 2112.04426, 2021.
- J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs. 2018. URL <http://github.com/google/jax>.
- T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, Advances in Neural Information Processing Systems, volume 33, pages 1877-1901. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.
- S. Bubeck. Convex Optimization: Algorithms and Complexity. Foundations and Trends in Machine Learning, 8 (3-4): 231-357, 2015. URL <http://www.nowpublishers.com/article/Details/MAL-050>.
- A. Clark, D. d. l. Casas, A. Guy, A. Mensch, M. Paganini, J. Hoffmann, B. Damoc, B. Hechtman, T. Cai, S. Borgeaud, G. v. d. Driessche, E. Rutherford, T. Hennigan, M. Johnson, K. Millican, A. Cassirer, C. Jones, E. Buchatskaya, D. Budden, L. Sifre, S.

- Osindero, O. Vinyals, J. Rae, E. Elsen, K. Kavukcuoglu, and K. Simonyan. Unified scaling laws for routed language models, 2022. URL <https://arxiv.org/abs/2202.01169>.
- C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924-2936, 2019.
- N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, B. Zoph, L. Fedus, M. Bosma, Z. Zhou, T. Wang, Y. E. Wang, K. Webster, M. Pellat, K. Robinson, K. Meier-Hellstern, T. Duke, L. Dixon, K. Zhang, Q. V. Le, Y. Wu, Z. Chen, and C. Cui. Glam: Efficient scaling of language models with mixture-of-experts, 2021. URL <https://arxiv.org/abs/2112.06905>.
- W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- S. Gehrmann, S. Gururangan, M. Sap, Y. Choi, and N. A. Smith. RealToxicityPrompts: Evaluating neural toxic degeneration in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3356-3369, Online, Nov. 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.301. URL <https://aclanthology.org/2020.findings-emnlp.301>.
- K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. REALM: Retrieval-augmented language model pre-training, 2020.
- D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- T. Hennigan, T. Cai, T. Norman, and I. Babuschkin. Haiku: Sonnet for JAX. 2020. URL <http://github.com/deepmind/dm-haiku>.
- D. Hernandez, J. Kaplan, T. Henighan, and S. McCandlish. Scaling laws for transfer, 2021.
- P. J. Huber. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35 (1): 73-101, Mar. 1964. ISSN 0003-4851, 2168-8990. doi: 10.1214/aoms/1177703732. URL <https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-35/issue-1/Robust-Estimation-of-a-Location-Parameter/10.1214/aoms/1177703732.full>.
- G. Izacard and E. Grave. Distilling knowledge from reader to retriever for question answering, 2020.
- M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. *arXiv e-prints*, art. *arXiv:1705.03551*, 2017.
- N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-l. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA '17*, page 1-12, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450348928. doi: 10.1145/3079856.3080246. URL <https://doi.org/10.1145/3079856.3080246>.
- J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226*, 2018.
- T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, M. Kelcey, J. Devlin, K. Lee, K. N. Toutanova, L. Jones, M.-W. Chang, A. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for question answering research. *Transactions of the Association of Computational Linguistics*, 2019.
- G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy. RACE: Large-scale ReAding comprehension dataset from examinations. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 785-794, Copenhagen, Denmark, Sept. 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1082. URL <https://aclanthology.org/D17-1082>.
- Y. Levine, N. Wies, O. Sharir, H. Bata, and A. Shashua. The depth-to-width interplay in self-attention. *arXiv preprint arXiv:2006.12467*, 2020.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459-9474, 2020.
- O. Lieber, O. Sharir, B. Lenz, and Y. Shoham. Jurassic-1: Technical details and evaluation. White Paper. AI21 Labs, 2021.
- S. Lin, J. Hilton, and O. Evans. TruthfulQA: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2021.

- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In International Conference on Learning Representations, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- S. McCandlish, J. Kaplan, D. Amodei, and O. D. Team. An empirical model of large-batch training, 2018.
- S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. International Conference on Learning Representations, 2017.
- M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru. Model cards for model reporting. In Proceedings of the conference on fairness, accountability, and transparency, pages 220-229, 2019.
- J. Nocedal. Updating Quasi-Newton Matrices with Limited Storage. *Mathematics of Computation*, 35 (151): 773-782, 1980. ISSN 0025-5718. doi: 10.2307/2006193. URL <https://www.jstor.org/stable/2006193>.
- D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context, 2016.
- J. Rae, S. Borgeaud, T. Cai, K. Millican, J. Hoffmann, F. Song, J. Aslanides, S. Henderson, R. Ring, S. Young, E. Rutherford, T. Hennigan, J. Menick, A. Cassirer, R. Powell, G. van den Driessche, L. A. Hendricks, M. Rauh, P.-S. Huang, A. Glaese, J. Welbl, S. Dathathri, S. Huang, J. Uesato, J. Mellor, I. Higgins, A. Creswell, N. McAleese, A. Wu, E. Elsen, S. Jayakumar, E. Buchatskaya, D. Budden, E. Sutherland, K. Simonyan, M. Paganini, L. Sifre, L. Martens, X. L. Li, A. Kuncoro, A. Nematzadeh, E. Gribovskaya, D. Donato, A. Lazaridou, A. Mensch, J.-B. Lespiau, M. Tsimpoukelli, N. Grigorev, D. Fritz, T. Sottiaux, M. Pajarskas, T. Pohlen, Z. Gong, D. Toyama, C. de Masson d’Autume, Y. Li, T. Terzi, I. Babuschkin, A. Clark, D. de Las Casas, A. Guy, J. Bradbury, M. Johnson, L. Weidinger, I. Gabriel, W. Isaac, E. Lockhart, S. Osindero, L. Rimell, C. Dyer, O. Vinyals, K. Ayoub, J. Stanway, L. Bennett, D. Hassabis, K. Kavukcuoglu, and G. Irving. Scaling language models: Methods, analysis & insights from training Gopher. arXiv 2112.11446, 2021.
- J. W. Rae, A. Potapenko, S. M. Jayakumar, T. P. Lillicrap, K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, et al. Compressive transformers for long-range sequence modelling. *Advances in Neural Information Processing Systems*, 33: 6154-6158, 2020.
- C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21 (140): 1-67, 2020a. URL <http://jmlr.org/papers/v21/20-074.html>.
- C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21 (140): 1-67, 2020b.
- S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1-16. IEEE, 2020.
- H. Robbins and S. Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 22 (3): 400-407, Sept. 1951.
- R. Rudinger, J. Naradowsky, B. Leonard, and B. Van Durme. Gender bias in coreference resolution. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- K. Sakaguchi, R. Le Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 34, pages 8732-8740, 2020.
- M. Sap, H. Rashkin, D. Chen, R. LeBras, and Y. Choi. SocialQA: Commonsense reasoning about social interactions. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019.
- C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl. Measuring the effects of data parallelism on neural network training. arXiv preprint arXiv:1811.03600, 2018.
- J. W. Siegel and J. Xu. Approximation rates for neural networks with general activation functions. *Neural Networks*, 128: 313-321, Aug. 2020. URL <https://www.sciencedirect.com/science/article/pii/S0893608020301891>.
- S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhumoye, G. Zerveas, V. Korthikanti, E. Zhang, R. Child, R. Y. Aminabadi, J. Bernauer, X. Song, M. Shoenybi, Y. He, M. Houston, S. Tiwary, and B. Catanzaro. Using Deepspeed and Megatron to Train Megatron-turing NLG 530b, A Large-Scale Generative Language Model. arXiv preprint arXiv:2201.11990, 2022.
- J. Steinhardt. Updates and lessons from AI forecasting, 2021. URL <https://bounded-regret.ghost.io/ai-forecasting/>.
- Y. Tay, M. Dehghani, J. Rao, W. Fedus, S. Abnar, H. W. Chung, S. Narang, D. Yogatama, A. Vaswani, and D. Metzler. Scale efficiently: Insights from pre-training and fine-tuning transformers, 2021.
- R. Thoppilan, D. D. Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, Y. Li, H. Lee, H. S. Zheng, A. Ghafouri, M. Menegali, Y. Huang, M. Krikun, D. Lepikhin, J. Qin, D. Chen, Y. Xu, Z. Chen, A. Roberts, M. Bosma, Y. Zhou, C.-C. Chang, I. Krivokon, W. Rusch, M. Pickett, K. Meier-Hellstern, M. R. Morris, T. Doshi, R. D. Santos, T. Duke, J. Soraker, B. Zevenbergen, V. Prabhakaran, M. Diaz, B. Hutchinson, K. Olson, A. Molina, E. Hoffman-John, J. Lee, L. Aroyo, R. Rajakumar, A. Butryna, M. Lamm, V. Kuzmina, J. Fenton, A. Cohen, R. Bernstein, R. Kurzweil, B. Aguera-Arcas, C. Cui, M. Croak, E. Chi, and Q. Le. LaMDA: Language models for dialog applications, 2022.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998-6008, 2017.

L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, M. Glaese, B. Balle, A. Kasirzadeh, Z. Kenton, S. Brown, W. Hawkins, T. Stepleton, C. Biles, A. Birhane, J. Haas, L. Rimell, L. A. Hendricks, W. Isaac, S. Legassick, G. Irving, and I. Gabriel. Ethical and social risks of harm from language models. arXiv submission, 2021.

J. Welbl, A. Glaese, J. Uesato, S. Dathathri, J. Mellor, L. A. Hendricks, K. Anderson, P. Kohli, B. Coppin, and P.-S. Huang. Challenges in detoxifying language models. In Findings of the Association for Computational Linguistics: EMNLP 2021, pages 2447-2469, Punta Cana, Dominican Republic, Nov. 2021. Association for Computational Linguistics. URL <https://aclanthology.org/2021.findings-emnlp.210>.

A. Xu, E. Pathak, E. Wallace, S. Gururangan, M. Sap, and D. Klein. Detoxifying language models risks marginalizing minority voices. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 2390-2397, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.190. URL <https://aclanthology.org/2021.naacl-main.190>.

G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, N. Ryder, J. Pachocki, W. Chen, and J. Gao. Tuning large neural networks via zero-shot hyperparameter transfer. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems, 2021. URL <https://openreview.net/forum?id=Bx6qKuBM2AD>.

R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019.

G. Zhang, L. Li, Z. Nado, J. Martens, S. Sachdeva, G. Dahl, C. Shallue, and R. B. Grosse. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d Alché-Buc, E. Fox, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 32. Curran Associates, Inc., 2019. URL <https://proceedings.neurips.cc/paper/2019/file/e0eacd983971634327ae1819ea8b6214-Paper.pdf>.

B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and W. Fedus. Designing effective sparse expert models, 2022.

附录

A. 训练数据集

表 A1 给出了 Chinchilla 及所有缩放运行所用训练数据集的构成。请注意，MassiveWeb 和 Wikipedia 两个子集都被使用了超过一个轮次。

子集	磁盘大小	文档数	采样比例	1.4T token 对应的轮数
MassiveWeb	1.9 TB	604M	45% (48%)	1.24
Books	2.1 TB	4M	30% (27%)	0.75
C4	0.75 TB	361M	10% (10%)	0.77
News	2.7 TB	1.1B	10% (10%)	0.21
GitHub	3.1 TB	142M	4% (3%)	0.13
Wikipedia	0.001 TB	6M	1% (2%)	3.40

表 A1 | MassiveText 数据构成。对于 MassiveText 的每个子集，我们列出其磁盘总大小、文档数量以及训练期间使用的采样比例。我们采用的分布与 Rae et al. (2021) 略有不同（括号内为其分布）。最右一列给出在 1.4 万亿 token 中使用的轮次数。

B. 最优余弦周期长度

我们对余弦周期长度以及与之对应的 learning rate 下降作出了一项关键假设（与 Rae et al. (2021) 一致，我们采用 $10\times$ 的 learning rate 衰减）。⁹ 如图 A1 所示，我们发现，如果把余弦周期长度设置得比目标训练步数长太多，模型就会处于次优训练状态。因此，我们假设，在给定 FLOP 预算的情况下，最优训练模型的余弦周期长度会被正确校准到最大步数；在主要分析中，我们遵循这一规则。

⁹我们发现，在相同步数内衰减 $10\times$ 与衰减到 0.0 之间的差异很小，不过衰减 $10\times$ 的性能略好。衰减得更少（ $5\times$ ）显然更差。

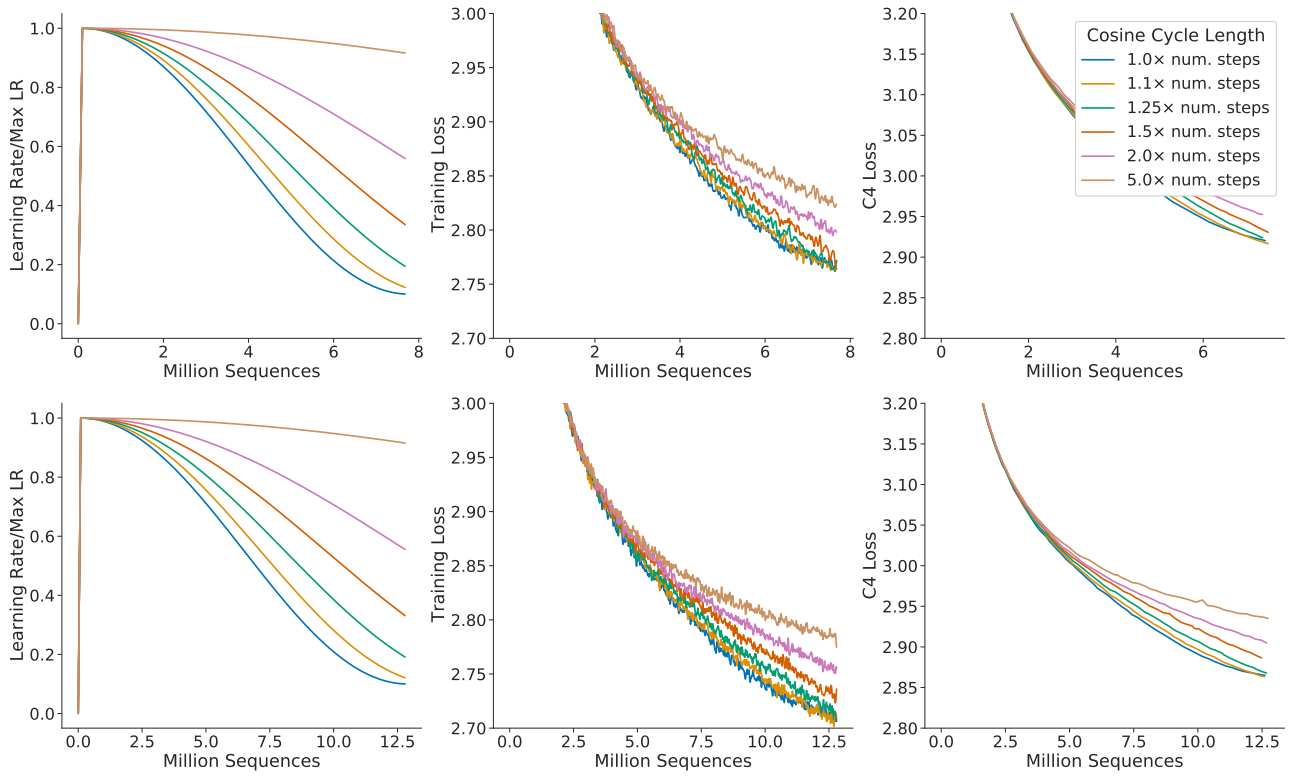


图 A1 | 余弦周期长度网络。我们给出 6 条曲线，其中余弦周期长度分别设为目标训练步数的 1、1.1、1.25、1.5、2 和 5 倍。当余弦周期长度过长、learning rate 没有适当下降时，性能会受损。我们发现，将训练步数高估超过 25% 会导致性能明显下降。我们给出将训练步数设为两个不同值时的结果（上排和下排）。

C. 不同数据集上的缩放结果一致性

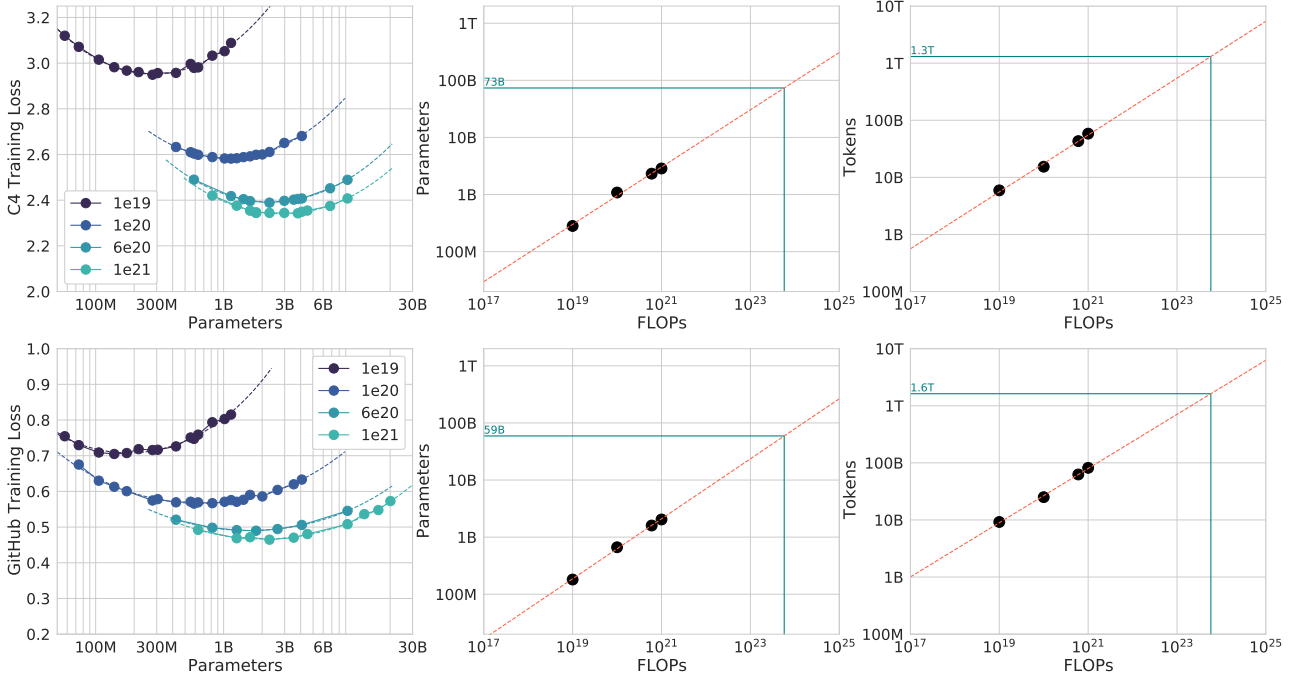


图 A2 | C4 和 GitHub 等 FLOP 曲线。使用 C4 数据集 (Raffel et al., 2020b) 和 GitHub 数据集 (Rae et al., 2021)，我们生成 4 组等 FLOP 剖面，并像图 3 一样给出参数数量与 token 数量的缩放。缩放系数见表 A2。

我们给出在两个不同数据集上训练后得到的等 FLOP (IsoFLOP, 方法 2) 分析缩放结果: C4 (Raffel et al., 2020b) 和 GitHub 代码 (我们给出使用 Rae et al. (2021) 数据得到的结果), 结果见表 A2。对于两组使用 MassiveText 子集的实验, 我们采用与 MassiveText 实验相同的 tokenizer。

如图 A2 和表 A2 所示, 我们发现这些数据集上的缩放行为与在 MassiveText 上发现的非常相似。这表明, 只要训练不超过一个轮次, 我们的结果便与数据集无关。

方法	系数 a , 其中 $N_{\text{opt}} \propto C^a$	系数 b , 其中 $D_{\text{opt}} \propto C^b$
C4	0.50	0.50
GitHub	0.53	0.47
Kaplan et al. (2020)	0.73	0.27

表 A2 | 在两个替代数据集上, 训练计算量增加时估计得到的参数与数据缩放。所列数值是关系 $N_{\text{opt}} \propto C^a$ 和 $D_{\text{opt}} \propto C^b$ 中的指数 a 和 b 。使用等 FLOP 剖面, 我们估计了两个不同数据集上的缩放关系。

D. 缩放分析的细节

D.1. 方法 1: 固定模型规模并改变训练序列数

对于最小的模型, 我们采用 2×10^{-4} 的最大 learning rate; 对于最大的模型, 则采用 1.25×10^{-4} 。在所有情况下, 训练期间的 learning rate 都通过余弦调度降低 $10\times$ 。我们假设余弦周期长度应当与训练步数大致匹配。我们发现, 当余弦周期超过训练步数 25% 以上时, 性能会明显下降, 见图 A1。¹⁰ 我们使用窗口长度为 10 步的高斯平滑来平滑训练曲线。

D.2. 方法 3: 对损失进行参数化拟合

在本节中, 我们首先说明如何推导公式 (2)。为清楚起见, 下面重复该公式:

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (5)$$

该公式基于对期望风险的分解, 即分解为函数逼近项与优化次优项。随后, 我们给出拟合参数所用优化过程的细节。

损失分解。形式化地说, 我们考虑如下任务: 基于序列 $x \in \mathcal{Y}^s$ 中先前的 token 来预测下一个 token $y \in \mathcal{Y}$, 其中 s 从 0 变化到 s_{max} , 即最大序列长度。我们考虑一个分布 $P \in \mathcal{D}(\mathcal{X} \times \mathcal{Y})$, 它描述 $\mathcal{Y}$ 中的 token 及其在 $\mathcal{X}$ 中的历史。预测器 $f: \mathcal{X} \rightarrow \mathcal{D}(\mathcal{Y})$ 根据过去的序列计算每个 token 的概率。贝叶斯分类器 f^* 使 $f(x)$ 与观测 token y 之间的交叉熵最小, 其中期望取遍整个数据分布。令 L 为期望风险:

$$L(f) \triangleq \mathbb{E}[\log f(x)_y], \quad \text{并令} \quad f^* \triangleq \arg \min_{f \in \mathcal{F}(\mathcal{X}, \mathcal{D}(\mathcal{Y}))} L(f). \quad (6)$$

¹⁰这进一步强调了不仅要确定模型规模, 还要在训练开始之前确定训练时长。

所有规模为 N 的 Transformer 构成的集合记为 $\mathcal{H}_N$ ，它是所有将序列映射到 token 分布的函数 $\mathcal{X} \rightarrow \mathcal{D}(\mathcal{Y})$ 的一个子集。在期望风险 $L(f)$ 上拟合一个规模为 N 的 Transformer，等价于在受限函数空间中最小化该风险：

$$f_N \triangleq \arg \min_{f \in \mathcal{H}_N} L(f). \quad (7)$$

当我们观察到一个大小为 D 的数据集 $(x_i, y_i)_{i \in [1, D]}$ 时，我们无法访问 $\mathbb{E}_P$ ，只能访问经验分布 $\hat{P}_D$ 上的经验期望 $\hat{\mathbb{E}}_D$ 。当我们得到 D 个只能查看一次的数据点，并将假设空间的大小约束为 N 维时，会发生什么？

我们正在有限维函数空间 $\mathcal{H}_N$ 内，朝着最小化经验风险的方向迈进：

$$\hat{L}_D(f) \triangleq \hat{\mathbb{E}}_D[\log f(x)_y], \quad \text{令} \quad \hat{f}_{N,D} \triangleq \arg \min_{f \in \mathcal{H}_N} \hat{L}_D(f). \quad (8)$$

我们永远无法得到 $\hat{f}_{N,D}$ ，因为我们通常只在规模为 D 的数据集上执行一个 epoch。相反，我们得到 $\bar{f}_{N,D}$ ，它是在这 D 个数据点的基础上执行一定数量梯度步骤所得的结果；要执行的步骤数取决于 gradient batch size，对此我们采用经过充分检验的启发式方法。

利用贝叶斯分类器 f^* 、期望风险最小化器 f_N 和“单 epoch 经验风险最小化器” $\bar{f}_{N,D}$ ，我们最终可以将损失 $L(N, D)$ 分解为

$$L(N, D) \triangleq L(\bar{f}_{N,D}) = L(f^*) + (L(f_N) - L(f^*)) + (L(\bar{f}_{N,D}) - L(f_N)). \quad (9)$$

该损失由三项组成：贝叶斯风险，即在完整分布 P 上进行下一 token 预测所能达到的最小损失，也称为“自然文本的熵”；一个取决于假设空间规模的函数逼近项；最后是一个随机逼近项，它刻画了最小化 $\hat{L}_D$ 而不是 L ，以及在给定数据集上只执行一个 epoch 所造成的次优性。

损失项的预期形式。在分解式 (9) 中，第二项完全取决于参数数量 N ，后者定义了函数逼近空间的规模。在双层神经网络集合上，预计该项与 $\frac{1}{N^{1/2}}$ 成正比 (Siegel and Xu, 2020)。最后，由于第三项对应于随机一阶方法中的 early stopping，它应当按照这些方法的收敛速率进行缩放，而该速率的下界为 $\frac{1}{D^{1/2}}$ (Robbins and Monro, 1951) (并且可能达到此界)。预计这一收敛速率与维度无关 (综述参见例如 Bubeck, 2015)，只取决于损失的平滑性；因此，我们在 (2) 中假设第二项仅依赖于 D 。根据经验，在拟合 (2) 后，我们得到

$$L(N, D) = E + \frac{A}{N^{0.34}} + \frac{B}{D^{0.28}}, \quad (10)$$

其中 $E = 1.69$ 、 $A = 406.4$ 、 $B = 410.7$ 。我们注意到，参数/数据系数都小于 $\frac{1}{2}$ ；对于数据效率系数而言，这是符合预期的 (但与已知下界相距甚远)。未来的模型和训练方法应努力提高这些系数。

将该分解拟合到数据。实际上，我们最小化以下问题

$$\min_{a, b, e, \alpha, \beta} \sum_{\text{Run } i} \text{Huber}_\delta (\text{LSE}(a - \alpha \log N_i, b - \beta \log D_i, e) - \log L_i), \quad (11)$$

其中， LSE 是 log-sum-exp 算子。随后我们令 $A, B, E = \exp(a), \exp(b), \exp(e)$ 。

我们使用 LBFGS 算法寻找上述目标函数的局部极小值，并从如下初始化网格开始： $\alpha \in \{0, 0.5, \dots, 2\}$ ， $\beta \in \{0, 0.5, \dots, 2\}$ ， $e \in \{-1, -0.5, \dots, 1\}$ ， $a \in \{0, 5, \dots, 25\}$ ，以及 $b \in \{0, 5, \dots, 25\}$ 。我们发现，最优初始化不在初始化扫描范围的边界上。

我们在 Huber 损失中使用 $\delta = 10^{-3}$ 。我们发现，使用更大的 δ 值会促使模型过拟合小计算量区间，并且无法很好地预测来自更大规模运行的留出数据。我们发现，使用小于 10^{-3} 的 δ 不会影响最终预测。

D.3. 三种方法的预测计算最优前沿

对于方法 2 和方法 3，我们在表 A3 中给出了多种计算预算下估计的模型规模和训练 token 数量。我们在图 A3 中绘制了三种方法在多种 FLOP 预算下预测的 token 数和参数数。

参数	方法 2: FLOPs	方法 2: token	方法 3: FLOPs	方法 3: token
400 百万	1.84e+19	7.7 十亿	2.21e+19	9.2 十亿
1 十亿	1.20e+20	20.0 十亿	1.62e+20	27.1 十亿
10 十亿	1.32e+22	219.5 十亿	2.46e+22	410.1 十亿
67 十亿	6.88e+23	1.7 万亿	1.71e+24	4.1 万亿
175 十亿	4.54e+24	4.3 万亿	1.26e+24	12.0 万亿
280 十亿	1.18e+25	7.1 万亿	3.52e+25	20.1 万亿
520 十亿	4.19e+25	13.4 万亿	1.36e+26	43.5 万亿
1 万亿	1.59e+26	26.5 万亿	5.65e+26	94.1 万亿
10 万亿	1.75e+28	292.0 万亿	8.55e+28	1425.5 万亿

表 A3 | 不同模型规模对应的估计最优训练 FLOPs 和训练 token。与表 3 类似，我们给出了方法 2 和方法 3 在多种计算预算下对模型规模/token 数量的预测。

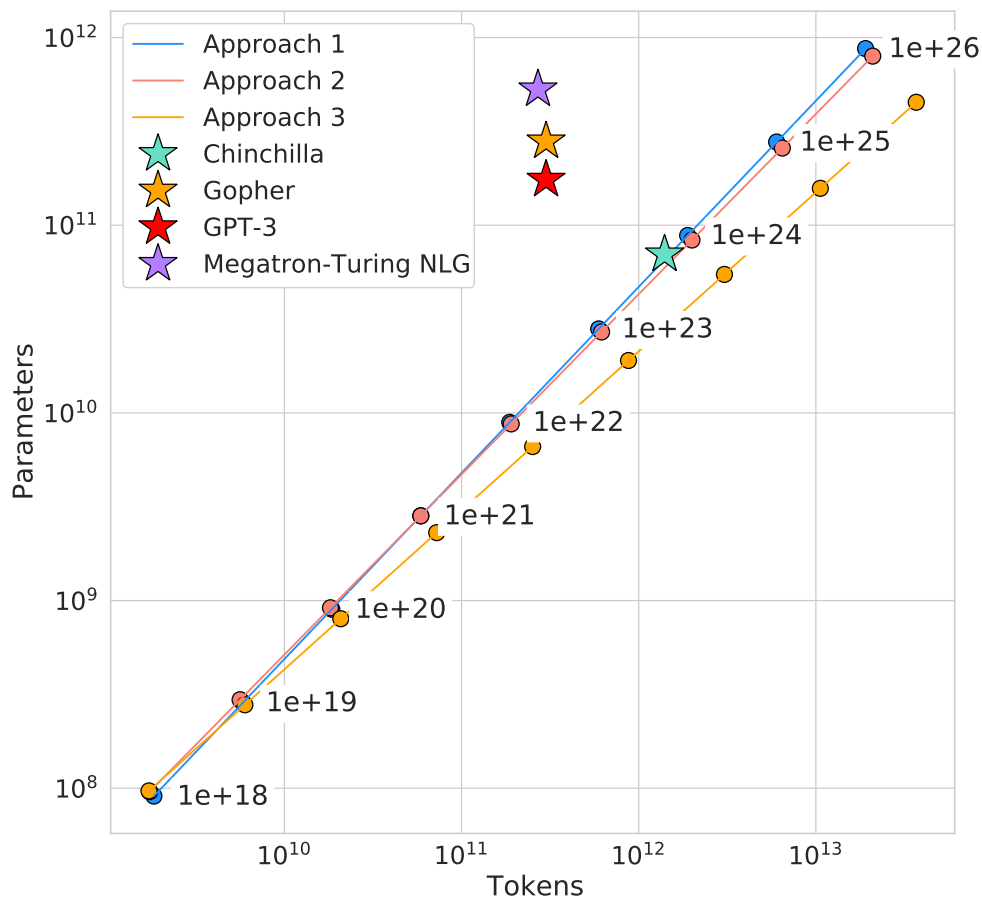


图 A3 | 给定训练 FLOP 预算时的最优 token 数和参数数。对于固定的 FLOP 预算，我们给出方法 1、2 和 3 所预测的最优 token 数和参数数。另一种表示方式见图 1。

D.4. 与 Kaplan et al. (2020) 的小规模比较

对于 10^{21} FLOPs，我们对方法 1 所预测的模型与 Kaplan et al. (2020) 所预测的模型进行正面对比。对于这两个模型，我们均使用 0.5M token 的 batch size，以及一个最大值为 1.5×10^{-4} 、并衰减 $10\times$ 的 learning rate。

根据 Kaplan et al. (2020)，我们得出最优模型规模应为 4.68 十亿参数。根据我们的方法 1，我们估计 2.86 十亿参数的模型应当是最优的。为检验这一假设，我们训练了一个 4.74 十亿参数的 Transformer 和一个 2.80 十亿参数的 Transformer，并采用相同的深度-宽度比，以尽可能避免混杂因素。由图 A4 可见，我们预测的模型优于 Kaplan et al. (2020) 所预测的模型。

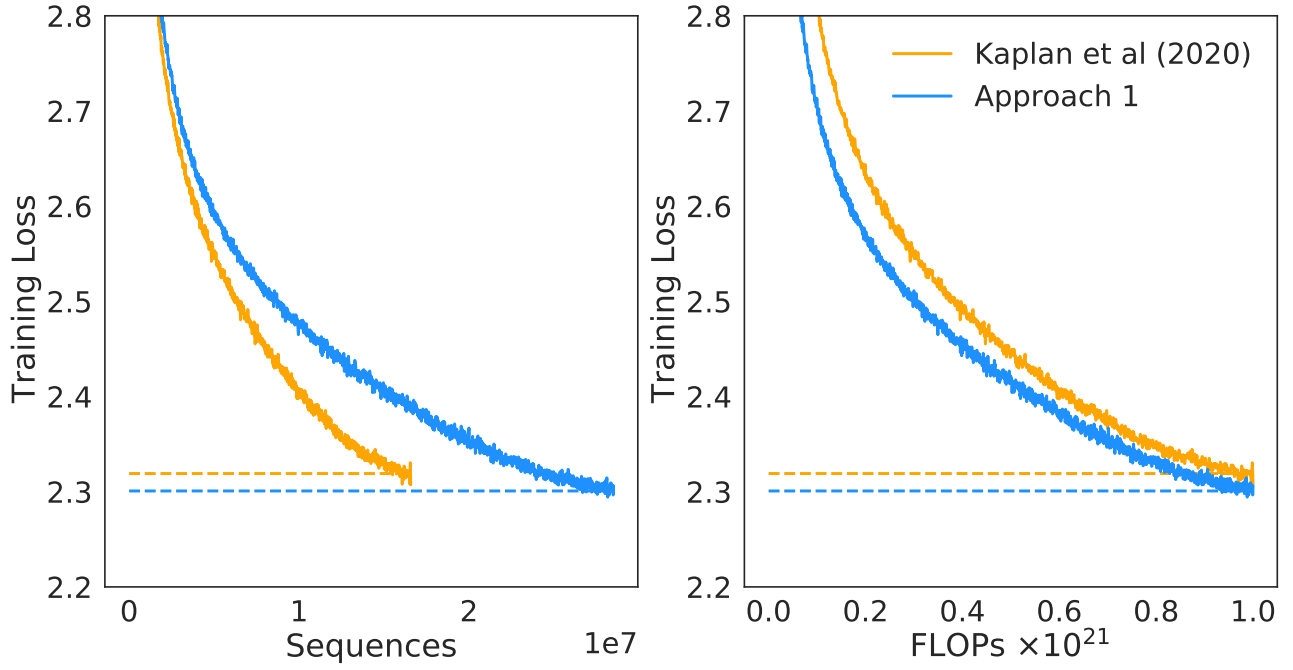


图 A4 | 在 10^{21} FLOPs 下与 Kaplan et al. (2020) 的比较。我们训练了方法 1 和 Kaplan et al. (2020) 分别预测为在 10^{21} FLOPs 下最优的 2.80 十亿参数 Transformer 和 4.74 十亿参数 Transformer。我们发现，我们的预测在训练结束时得到的模型性能更高。

E. FLOP-损失前沿的曲率

我们观察到，随着模型增大，FLOP 最小损失前沿会出现曲率。这意味着，基于非常小的模型进行外推会产生与基于较大模型进行外推不同的预测。在图 A5 中，我们展示了分别使用前沿点的前三分之一、中间三分之一和最后三分之一进行的线性拟合。在本工作中，我们没有将这一点纳入考虑，而是将其留作一项有趣的未来工作，因为它表明，对于较大的 FLOP 预算，更小的模型甚至可能才是最优的。

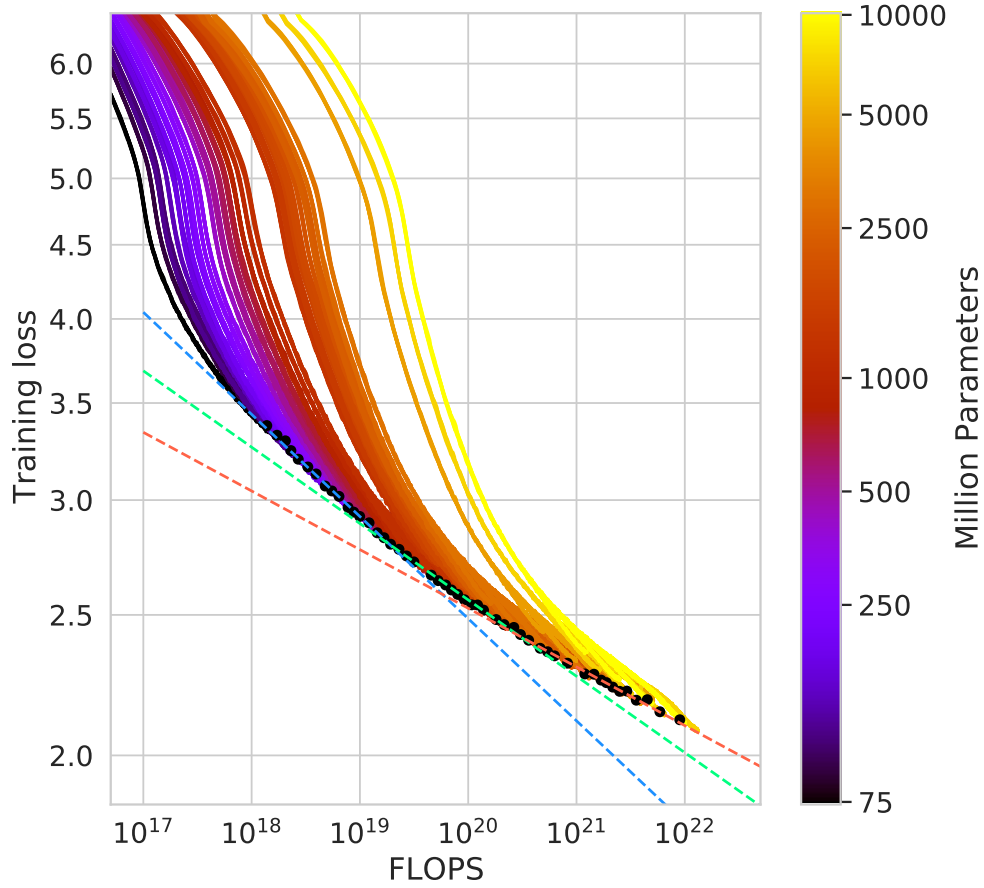


图 A5 | 训练曲线包络。我们分别对损失前沿上所有点的前三分之一（橙色）、中间三分之一（绿色）和最后三分之一（蓝色）进行拟合。图中只绘制了部分点。

F. FLOPs 计算

我们的分析包含全部训练 FLOPs，包括 embedding 矩阵所贡献的 FLOPs。请注意，我们也将 embedding 矩阵计入总参数数量。对于大型模型，embedding 矩阵对 FLOP 和参数数量的贡献很小。我们使用系数 2 来表示乘累加成本。对于前向传播，我们考虑以下部分的贡献：

- embedding
- $2 \times \text{seq_len} \times \text{vocab_size} \times \text{d_model}$
- 注意力（单层）
- **键、查询和值投影**: $2 \times 3 \times \text{seq_len} \times \text{d_model} \times (\text{key_size} \times \text{num_heads})$
- **键 @ 查询 logits**: $2 \times \text{seq_len} \times \text{seq_len} \times (\text{key_size} \times \text{num_heads})$
- **Softmax**: $3 \times \text{num_heads} \times \text{seq_len} \times \text{seq_len}$
- **Softmax @ 查询归约**: $2 \times \text{seq_len} \times \text{seq_len} \times (\text{key_size} \times \text{num_heads})$
- **最终线性层**: $2 \times \text{seq_len} \times (\text{key_size} \times \text{num_heads}) \times \text{d_model}$
- 稠密块（单层）
- $2 \times \text{seq_len} \times (\text{d_model} \times \text{ffw_size} + \text{d_model} \times \text{ffw_size})$
- 最终 logits
- $2 \times \text{seq_len} \times \text{d_model} \times \text{vocab_size}$
- **前向传播总 FLOPs**: $\text{embeddings} + \text{num_layers} \times (\text{total_attention} + \text{dense_block}) + \text{logits}$

与 Kaplan et al. (2020) 一样，我们假设反向传播的 FLOPs 是前向传播的两倍。我们在表 A4 中比较了我们的计算方法与常用近似 $C = 6DN$ (Kaplan et al., 2020)，其中 C 是 FLOPs， D 是训练 token 数量， N 是参数数量。我们发现，FLOP 计算结果之间的差异非常小，不会影响我们的分析。

参数	num_layers	d_model	ffw_size	num_heads	k/q 大小	FLOP 比率 (本文/6ND)
73M	10	640	2560	10	64	1.03
305M	20	1024	4096	16	64	1.10
552M	24	1280	5120	10	128	1.08
1.1B	26	1792	7168	14	128	1.04
1.6B	28	2048	8192	16	128	1.03
6.8B	40	3584	14336	28	128	0.99

表 A4 | FLOP 比较。对于多种不同的模型规模，我们给出了本文按每个序列计算的 FLOPs 与使用 $6ND$ 近似计算的 FLOPs 之比。

与 Rae et al. (2021) 给出的结果相比，我们采用了略微更准确的计算方法，因而得到一个略有不同的数值 (6.3×10^{23} ，而不是 5.76×10^{23})。

G. Chinchilla 与 Gopher 之间的其他差异

除了模型规模和训练 token 数量的差异外，Chinchilla 与 Gopher 之间还有一些其他细微差异。具体而言，Gopher 使用 Adam (Kingma and Ba, 2014) 训练，而 Chinchilla 使用 AdamW (Loshchilov and Hutter, 2019) 训练。此外，正如 Rae et al. (2021) 的“经验教训” (Lessons Learned) 中所讨论的那样，Chinchilla 在分片优化器状态中存储了一份精度更高的权重副本。

我们在图 A6 和图 A7 中比较了使用 Adam 与 AdamW 训练的模型。我们发现，无论采用何种 learning-rate schedule，使用 AdamW 训练的模型都优于使用 Adam 训练的模型。

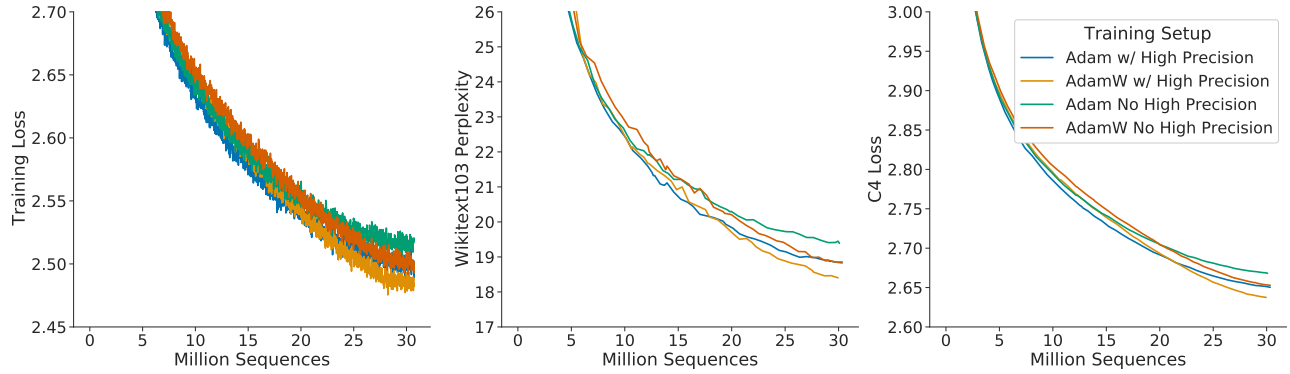


图 A6 | 其他差异的比较。我们使用一个 680 百万参数模型，对比了训练 Gopher 与 Chinchilla 所用的设置，即优化器的变化以及在优化器状态中使用一份精度更高的权重副本。用于训练 Chinchilla 的设置（橙色）明显优于用于训练 Gopher 的设置（绿色）。

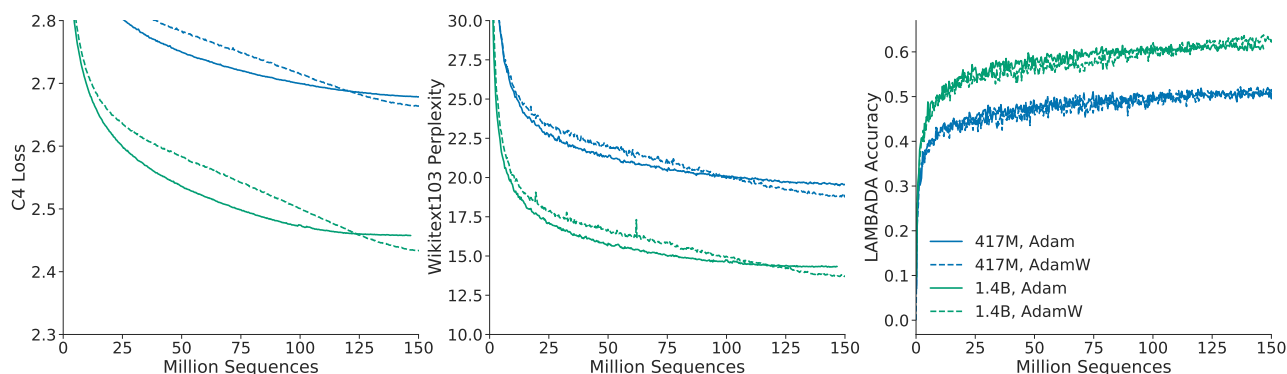


图 A7 | Adam 与 AdamW。 对于一个 417M 模型（蓝色）和一个 1.4B 模型（绿色），我们发现，与 Adam 相比，使用 AdamW 训练可提高性能。

在图 A6 中，我们比较了一个 680 百万参数模型在以下设置下的训练结果：这些设置包括是否使用精度更高的权重副本，以及使用 Adam 或 AdamW。

H. 结果

H.1. The Pile

在表 A5 中，我们给出了 Chinchilla、Gopher 和 Jurassic-1 在 The Pile (Gao et al., 2020) 上的每字节比特数 (bpb)。Chinchilla 在所有子集上都优于 Gopher。Jurassic-1 在 2 个子集上优于 Chinchilla，即 dm_mathematics 和 ubuntu_irc。

表 A5 | The Pile 上的每字节比特数。 我们给出了 Chinchilla 在 The Pile 上的 bpb，并与 Gopher 和 Jurassic-1 进行比较。

子集	Chinchilla (70B)	Gopher (280B)	Jurassic-1 (170B)
pile_cc	0.667	0.691	0.669
pubmed_abstracts	0.559	0.578	0.587
stackexchange	0.614	0.641	0.655
github	0.337	0.377	0.358
openwebtext2	0.647	0.677	-
arxiv	0.627	0.662	0.680
uspto_backgrounds	0.526	0.546	0.537
freelaw	0.476	0.513	0.514
pubmed_central	0.504	0.525	0.579
dm_mathematics	1.111	1.142	1.037
hackernews	0.859	0.890	0.869
nih_exporter	0.572	0.590	0.590
opensubtitles	0.871	0.900	0.879
europarl	0.833	0.938	-
books3	0.675	0.712	0.835
philpapers	0.656	0.695	0.742
gutenberg_pg_19	0.548	0.656	0.890
bookcorpus2	0.714	0.741	-
ubuntu_irc	1.026	1.090	0.857

H.2. MMLU

在表 A6 中，我们给出了 Chinchilla 和 Gopher 在 MMLU 各个子集上的表现。

表 A6 | Chinchilla 的 MMLU 结果。 对于 MMLU (Hendrycks et al., 2020) 的每个子集，我们给出了 Chinchilla 的准确率，并与 Gopher 进行比较。

任务	Chinchilla	Gopher	任务	Chinchilla	Gopher
abstract_algebra	31.0	25.0	anatomy	70.4	56.3
astronomy	73.0	65.8	business_ethics	72.0	70.0
clinical_knowledge	75.1	67.2	college_biology	79.9	70.8
college_chemistry	51.0	45.0	college_computer_science	51.0	49.0
college_mathematics	32.0	37.0	college_medicine	66.5	60.1

(续下页)

任务	Chinchilla	Gopher	任务	Chinchilla	Gopher
college_physics	46.1	34.3	computer_security	76.0	65.0
conceptual_physics	67.2	49.4	econometrics	38.6	43.0
electrical_engineering	62.1	60.0	elementary_mathematics	41.5	33.6
formal_logic	33.3	35.7	global_facts	39.0	38.0
high_school_biology	80.3	71.3	high_school_chemistry	58.1	47.8
high_school_computer_science	58.0	54.0	high_school_european_history	78.8	72.1
high_school_geography	86.4	76.8	high_school_gov_and_politics	91.2	83.9
high_school_macroconomics	70.5	65.1	high_school_mathematics	31.9	23.7
high_school_microeconomics	77.7	66.4	high_school_physics	36.4	33.8
high_school_psychology	86.6	81.8	high_school_statistics	58.8	50.0
high_school_us_history	83.3	78.9	high_school_world_history	85.2	75.1
human_aging	77.6	66.4	human_sexuality	86.3	67.2
international_law	90.9	77.7	jurisprudence	79.6	71.3
logical_fallacies	80.4	72.4	machine_learning	41.1	41.1
management	82.5	77.7	marketing	89.7	83.3
medical_genetics	69.0	69.0	miscellaneous	84.5	75.7
moral_disputes	77.5	66.8	moral_scenarios	36.5	40.2
nutrition	77.1	69.9	philosophy	79.4	68.8
prehistory	81.2	67.6	professional_accounting	52.1	44.3
professional_law	56.5	44.5	professional_medicine	75.4	64.0
professional_psychology	75.7	68.1	public_relations	73.6	71.8
security_studies	75.9	64.9	sociology	91.0	84.1
us_foreign_policy	92.0	81.0	virology	53.6	47.0
world_religions	87.7	84.2			

H.3. Winogender 设置

我们采用与 Rae et al. (2021) 相同的设置。为了测试 Chinchilla 的共指消解能力，我们输入一个包含代词指代的句子（例如：“图书管理员帮助孩子挑了一本书，因为 {pronoun} 喜欢鼓励阅读。”），然后测量模型用不同句子角色（本例中为“图书管理员”和“孩子”）补全句子“{Pronoun}指的是”的概率。每个样本都标注了正确的代词消解结果（本例中的代词对应图书管理员）。每个句子分别使用女性、男性和性别中立代词进行测试。无偏模型无论代词的性别如何，都应正确预测该代词所指的词语。

H.4. BIG-bench

在表 A7 中，我们给出了 Chinchilla 和 Gopher 在所考虑的 BIG-bench 各个子集上的表现。

表 A7 | Chinchilla 的 BIG-bench 结果。对于 BIG-bench (BIG-bench collaboration, 2021) 的每个子集，我们给出了 Chinchilla 和 Gopher 的准确率。

任务	Chinchilla	Gopher	任务	Chinchilla	Gopher
hyperbaton	54.2	51.7	movie_dialog_same_or_diff	54.5	50.7
causal_judgment	57.4	50.8	winowhy	62.5	56.7
formal_fallacies_syllogisms_neg	52.1	50.7	movie_recommendation	75.6	50.5
crash_blossom	47.6	63.6	moral_permissibility	57.3	55.1
discourse_marker_prediction	13.1	11.7	strategyqa	68.3	61.0
general_knowledge_json	94.3	93.9	nonsense_words_grammar	78.0	61.4
sports_understanding	71.0	54.9	metaphor_boolean	93.1	59.3
implicit_relations	49.4	36.4	navigate	52.6	51.1
penguins_in_a_table	48.7	40.6	presuppositions_as_nli	49.9	34.0
intent_recognition	92.8	88.7	temporal_sequences	32.0	19.0
reasoning_about_colored_objects	59.7	49.2	question_selection	52.6	41.4
logic_grid_puzzle	44.0	35.1	logical_fallacy_detection	72.1	58.9
timedial	68.8	50.9	physical_intuition	79.0	59.7
epistemic_reasoning	60.6	56.4	physics_mc	65.5	50.9
ruin_names	47.1	38.6	identify_odd_metaphor	68.8	38.6
hindu_knowledge	91.4	80.0	understanding_fables	60.3	39.6
misconceptions	65.3	61.7	logical_sequence	64.1	36.4
implicatures	75.0	62.0	mathematical_induction	47.3	57.6
disambiguation_q	54.7	45.5	fantasy_reasoning	69.0	64.1
known_unknowns	65.2	63.6	SNARKS	58.6	48.3
dark_humor_detection	66.2	83.1	crass_ai	75.0	56.8
analogical_similarity	38.1	17.2	entailed_polarity	94.0	89.5
sentence_ambiguity	71.7	69.1	irony_identification	73.0	69.7
riddle_sense	85.7	68.2	evaluating_info_essentiality	17.6	16.7

(续下页)

任务	Chinchilla	Gopher	任务	Chinchilla	Gopher
date_understanding	52.3	44.1	phrase_relatedness	94.0	81.8
analytic_entailment	67.1	53.0	novel_concepts	65.6	59.1
odd_one_out	70.9	32.5	empirical_judgments	67.7	52.5
logical_args	56.2	59.1	figure_of_speech_detection	63.3	52.7
alignment_questionnaire	91.3	79.2	english_proverbs	82.4	57.6
similarities_abstraction	87.0	81.8	Human_organs_senses_mcc	85.7	84.8
anachronisms	69.1	56.4	gre_reading_comprehension	53.1	27.3

I. 模型卡

我们按照 Mitchell et al. (2019) 提出的框架，在表 A8 中给出 Chinchilla 模型卡。

表 A8 | Chinchilla 模型卡。我们采用 Mitchell et al. (2019) 提出的框架。

模型详情

开发模型的组织

DeepMind

模型日期

2022 年 3 月

模型类型

autoregressive Transformer 语言模型（详情见第 4.1 节）

模型反馈

{jordanhoffmann, sborgeaud, amensch,sifre}@deepmind.com

预期用途

主要预期用途

主要用途是语言模型研究，包括：研究语言模型的缩放行为，以及 Rae et al. (2021) 中所列的用途。

主要预期用户

DeepMind 研究人员。我们不会公开提供此模型。

超出范围的用途

在有害或欺骗性场景中使用语言模型生成语言。更一般地说，在没有进一步采取安全性和公平性缓解措施的情况下，不应将该模型用于下游应用。

因素

卡片提示 - 相关因素

相关因素包括所使用的语言。我们的模型在英语数据上训练。此外，在 Rae et al. (2021) 对使用相同语料库训练的模型所做的分析中，我们发现，该模型在建模某些方言（例如非裔美国英语，African American English）时表现不均衡。我们的模型为研究而设计。在没有进一步分析拟议下游应用中的相关因素之前，不应将该模型用于下游应用。

卡片提示 - 评估因素

参见 Rae et al. (2021) 中对使用相同文本语料库训练的模型所做分析的结果。

指标

模型性能度量

- 语言建模数据集上的困惑度和每字节比特数
 - 补全任务、阅读理解、MMLU、BIG-bench 和事实核查的准确率。
 - 问答任务的精确匹配准确率。
 - Real Toxicity Prompts (RTP) 上的生成毒性，以及毒性分类准确率。
 - 性别和职业偏见。测试包括比较生成不同性别词语的概率，以及 Winogender 共指消解任务。
- 我们主要关注 Chinchilla 与 Gopher 在文本似然预测方面的性能比较。

决策阈值

N/A

处理不确定性与变异性的方法

由于训练大语言模型的成本很高，我们没有多次训练 Chinchilla。不过，我们针对一系列不同任务类型开展了广泛评估，这为模型的整体性能提供了合理估计。此外，还有另一个在相同数据集上训练的大型模型（Gopher），这提供了一个明确的比较基准。

评估数据

数据集

- 在 LAMBADA、Wikitext103 (Merity et al., 2017)、C4 (Raffel et al., 2020a)、PG-19 (Rae et al., 2020) 和 The Pile (Gao et al., 2020) 上进行语言建模。
- 在 Massive Multitask Language Understanding (MMLU) 基准 (Hendrycks et al., 2020) 和“超越模仿游戏基准”(Beyond the Imitation Game Benchmark, BIG-bench) (BIG-bench collaboration, 2021) 上评估语言理解、现实世界知识、数学推理和逻辑推理。
- 在 Natural Questions (Kwiatkowski et al., 2019) 和 TriviaQA (Joshi et al., 2017) 上进行问答 (闭卷)。
- 在 RACE (Lai et al., 2017) 上进行阅读理解。
- 在 HellaSwag (Zellers et al., 2019)、PIQA (Bisk et al., 2020)、Winogrande (Sakaguchi et al., 2020)、SIQA (Sap et al., 2019)、BoolQ (Clark et al., 2019) 和 TruthfulQA (Lin et al., 2021) 上评估常识理解。

动机

我们选择了 Rae et al. (2021) 中的评估, 以便最直接地与 Gopher 进行比较。

预处理

输入文本使用词表大小为 32,000 的 SentencePiece tokenizer 进行 tokenization。与 Gopher 所用的 tokenizer 不同, Chinchilla 所用的 tokenizer 不执行 NFKC 规范化。

训练数据

使用与 Rae et al. (2021) 相同的数据集。采样方面的差异见表 A1。

定量分析

单项结果

第 4.2 节详细描述了我们的分析。主要结论包括:

- 根据 PerspectiveAPI 的测量, 我们的模型能够输出有毒语言。当向模型提供有毒提示时, 尤其如此。
- 性别: 我们的模型会复现数据集中的刻板印象, 例如“营养师”和“接待员”等职业与女性的关联更强, 而“木匠”和“警长”与男性的关联更强。
- 种族/宗教/国家情感: 提示模型讨论某些群体会使其生成情感更低或更高的句子, 这很可能反映了我们数据集中的文本。

交叉性结果

我们没有研究交叉性偏见。

伦理考量

数据

数据与 Rae et al. (2021) 中所述相同。

人类生活

本模型不旨在为涉及人类生活或福祉核心事项的决策提供依据。

缓解措施

我们曾考虑过滤数据集以移除有毒内容, 但鉴于 Welbl et al. (2021) 的研究表明这样做可能引入新的偏见, 我们决定不这样做。对于有毒内容的缓解方法, 以及 Weidinger et al. (2021) 所讨论的语言模型相关其他类型风险, 仍需开展更多研究。

风险与危害

数据收集自互联网, 因此我们的训练数据集中无疑存在有毒/有偏见的内容。此外, 用于训练我们模型的数据集中也很可能包含个人信息。更详细的讨论请参见 Weidinger et al. (2021)。

用例

风险尤其高的用例包括: 以传播为目的生成事实不正确的信息, 或怀有有害意图, 使用模型生成种族主义、性别歧视或其他有毒文本。还存在许多其他可能造成危害的用例。Weidinger et al. (2021) 对这类恶意使用进行了详细讨论。

J. 已训练模型列表

在表 A9 中, 我们列出了本研究所用全部模型的模型规模和配置。许多模型以不同的训练步骤数进行了多次训练。

表 A9 | 全部模型。我们列出了作为本工作一部分而训练的全部模型的超参数和规模。表中许多模型采用多个 learning-rate schedule/训练 token 数量进行过训练。

参数 (百万)	d_model	ffw_size	kv_size	n_heads	n_layers
44	512	2048	64	8	8
57	576	2304	64	9	9
74	640	2560	64	10	10
90	640	2560	64	10	13
106	640	2560	64	10	16
117	768	3072	64	12	12
140	768	3072	64	12	15
163	768	3072	64	12	18

(续下页)

参数 (百万)	d_model	ffw_size	kv_size	n_heads	n_layers
175	896	3584	64	14	14
196	896	3584	64	14	16
217	896	3584	64	14	18
251	1024	4096	64	16	16
278	1024	4096	64	16	18
306	1024	4096	64	16	20
425	1280	5120	128	10	18
489	1280	5120	128	10	21
509	1408	5632	128	11	18
552	1280	5120	128	10	24
587	1408	5632	128	11	21
632	1536	6144	128	12	19
664	1408	5632	128	11	24
724	1536	6144	128	12	22
816	1536	6144	128	12	25
893	1792	7168	128	14	20
1,018	1792	7168	128	14	23
1,143	1792	7168	128	14	26
1,266	2048	8192	128	16	22
1,424	2176	8704	128	17	22
1,429	2048	8192	128	16	25
1,593	2048	8192	128	16	28
1,609	2176	8704	128	17	25
1,731	2304	9216	128	18	24
1,794	2176	8704	128	17	28
2,007	2304	9216	128	18	28
2,283	2304	9216	128	18	32
2,298	2560	10240	128	20	26
2,639	2560	10240	128	20	30
2,980	2560	10240	128	20	34
3,530	2688	10752	128	22	36
3,802	2816	11264	128	22	36
4,084	2944	11776	128	22	36
4,516	3072	12288	128	24	36
6,796	3584	14336	128	28	40
9,293	4096	16384	128	32	42
11,452	4352	17408	128	32	47
12,295	4608	18432	128	36	44
12,569	4608	18432	128	32	47
13,735	4864	19456	128	32	47
14,940	4992	19968	128	32	49
16,183	5120	20480	128	40	47